

The Eval Loop

The Eval Loop

HOW TO MEASURE, DEBUG, AND
IMPROVE AI AGENTS IN PRODUCTION



Brenn Hill

Copyright © 2026 Brenn Hill. All rights reserved.

No part of this publication may be reproduced without prior written permission, except for brief quotations used in reviews, commentary, and uses permitted by law.

First edition, 2026. Published independently.

This book is provided for educational purposes. AI systems, benchmarks, products, and regulations change quickly. Readers remain responsible for evaluating practices in their own technical, legal, safety, and organizational contexts.

ISBN (Paperback): pending. ASIN (Kindle): pending.

looprails.com

CONTENTS

Preface 6

Demos Lie 10

Read the Failures 16

From Failure to Task 24

Four Kinds of Truth 32

The Grader Ladder 42

Executable Truth 50

Judgment and Judges 57

Reading Traces 64

Statistics Without the Lab Coat 75

The Two Loops 86

Release Gates 93

Production Is the Real Test 101

When Evals Fail 110

Owning the Loop 119

The First Thirty Days 130

The Loop Stays Open 140

Appendix A: Exercises 143

Appendix B: Templates 154

Appendix C: Case Study Field Guide 165

Appendix D: Statistical Recipes 174

Glossary 181

References 188

About the Author 193

Preface

This book began with a page of notes and an uncomfortable amount of recognition.

The page was Hamel Husain's guide to AI product engineering. Its argument was practical: look at what your system actually does, study the failures, build evaluations around the things that matter, and use those evaluations to make the product better [FOUND-01]. It described the work I kept seeing strong AI teams discover, often after first trying nearly everything else.

They tried the clever prompt.

They tried the larger model.

They added a second clever prompt explaining the first clever prompt.

They tried capital letters.

Capital letters have a distinguished history in software requirements. They are also not a measurement system.

What made products improve was a loop: observe, define, measure, change, and observe again. The loop sounds obvious when written in one sentence. It becomes less obvious on Tuesday afternoon, when the demo works, the launch date is Friday, and someone asks whether the team really needs to label twenty more conversations.

This book is about that Tuesday afternoon.

Maybe you are the engineer wondering whether the eval tests the outcome or merely the wording. Maybe you are the product manager trying to turn "make it more helpful" into something a team can act on. Domain experts will recognize the dangerous mistakes hiding inside fluent prose. Leaders may recognize the green dashboard, the unhappy users, and the creeping suspicion that the dashboard is measuring the dashboard.

The central idea is that an AI system contains two loops.

The **agent loop** does the work. It observes, reasons, calls tools, checks results, retries, stops, or escalates. It may run in seconds.

The **eval loop** improves the worker. People sample real outcomes, analyze errors, turn failures into tasks, construct graders, compare changes, and update the system. It may run over days or weeks.

The loops depend on each other. An agent loop without an eval loop can repeat mistakes at extraordinary speed. An eval loop disconnected from the agent loop becomes a benchmark program: impressive charts, limited influence, excellent snacks at the quarterly review.

The seam between them is where the useful artifacts live: task cases, traces, graders, release rules, and production telemetry. Build that seam well and product development becomes cumulative. Every important failure can become a permanent case. Every change can be compared on the same evidence. Every judge can be challenged. Every release can be discussed in terms more precise than “the vibes seem better.”

Four published cases run through the book.

SWE-bench gives us executable truth: agents patch real software repositories and tests decide whether the patch works. Its later auditing also gives us a humbling lesson—tests can be precise and still encode the wrong contract [SWE-01] [SWE-03].

τ -bench and **τ^3 -bench** give us environmental truth: a customer-service agent may claim it changed a reservation, but the database gets a vote [TAU-01] [TAU-02].

HealthBench gives us expert truth: physicians define case-specific criteria for what a strong response must notice, communicate, and avoid [HEALTH-01].

DeepResearch Bench gives us evidentiary truth: a polished report is not enough; its claims must be supported by the sources it cites [DRB-01].

Together they show that there is no universal grader. The right grader depends on where truth lives.

You will not find a catalog of evaluation tools here. Product interfaces age in dog years, and AI product interfaces may be dogs riding motorcycles. We will use small schemas and pseudocode, but our real subject is the durable work: choosing tasks, defining evidence, calibrating judgment, comparing noisy results, learning from production, and deciding who owns the loop.

I will not promise certainty, either. Evals are evidence, not a force field around production. A well-designed eval reduces uncertainty about a specific decision. A portfolio of offline tasks, deterministic checks, expert review, production monitoring, and controlled experiments reduces it further. An open system still refuses to become a theorem. Rude, but realistic.

What you get instead is disciplined learning. That is more useful than fake certainty anyway.

By the last page, you will know how to start with twenty real cases, build the strongest available grader for each, test the graders, compare variants honestly, connect offline results to production, and keep the whole contraption alive.

Source IDs such as [SWE-01] point to the references and to the complete editorial codex. The codex records citations, versions, claim boundaries, and reuse cautions. An eval book should probably keep an eval trail for its own claims. Otherwise the irony becomes structurally load-bearing.

Right. Let's get our hands dirty.

PART I



Discover

Demos Lie

Not maliciously. Demos are usually lovely people. They are simply answering a much easier question.

A demo asks, **Can the system succeed once, on a case we chose, while we are watching?**

A product asks:

- Can it succeed on cases users choose?
- Can it succeed repeatedly?
- Can it fail safely?
- Can we tell when it failed?
- Can the team improve it without breaking something else?

These questions are cousins, but they are not twins.

The familiar launch story starts with a striking result. Someone types a request. The agent searches, reasons, calls three tools, and produces an answer that would have taken a person an hour. People lean toward the screen. Someone says, “That is amazing.” It is amazing.

Then the product meets variation.

The next user is vague. The account has an old configuration. The policy changed last month. The relevant document is a scanned PDF with a table split across two pages. A tool times out after making the change but before returning confirmation. The model politely reports success. The database, showing a certain lack of team spirit, disagrees.

The demo was not fake. It was a sample of one.

THE THREE GAPS

AI products routinely fall into three gaps between demonstration and dependable use.

The task gap

The case in the demo is clean. Production cases come from a distribution nobody fully specified. Some are common; some are rare but costly; some are new combinations of old problems.

An eval suite begins by sampling that distribution intentionally. Represent the decision, not the universe. A team deciding whether to release a support agent needs ordinary refund requests, policy-edge cases, ambiguous requests, tool failures, and a small number of must-never-happen cases. A perfect ontology of human desire can wait. That project has been delayed.

The truth gap

The output looks right, but what would prove it?

For code, proof may be a test, a type check, a changed file, or an invariant. For a customer-service task, it may be the final database state plus policy compliance. For clinical guidance, it may require expert criteria. For research, it may be whether each substantive claim is supported by a source.

Fluency proves the output is fluent. Booked flights, fixed bugs, safe recommendations, and supported claims need evidence of their own.

The repetition gap

The system can do the task. Can it do the task reliably?

Suppose an agent succeeds on a task with probability 75 percent. If you give it three attempts and need only one success, the chance of at least one success is about 98.4 percent. A best-of-three demo looks terrific.

If a user needs the workflow to succeed three times in a row, the chance is about 42.2 percent. Same model. Same per-attempt score. Very different product.

τ -bench made this distinction concrete with pass^k : the probability that all k trials succeed. Its original retail results placed frontier agents below 25 percent on pass^8 , even where individual successes looked encouraging [TAU-01]. The metric asks the question a returning user asks: “Will this keep working?”

FOUR PRODUCTS, FOUR WAYS TO BE FOOLED

The cases we will follow each reveal a different version of demo success.

The patch that passes

In SWE-bench, an agent receives a real repository and GitHub issue, then writes a patch. Tests run. Green means success [SWE-01]. Reading the patch and admiring its posture is much weaker evidence.

But what if the hidden test requires a detail the issue never asked for? What if it accepts one narrow behavior but misses a broader regression? A later targeted audit of frequently failed SWE-bench Verified tasks reported material task or test problems in at least 59.4 percent of the 138 audited cases [SWE-03]. The number should not be extrapolated to the entire benchmark, but the lesson travels well: an executable grader can be wrong with tremendous precision.

The reservation that “changed”

In τ -bench, an agent interacts with a user, follows a policy, and calls tools in a simulated domain. The evaluator can inspect the final state. Did the reservation change? Was the fee correct? Did the agent make a prohibited action? Did it communicate the right facts? [TAU-03]

A transcript can sound successful while the environment remains untouched. It can also reach the right state by violating policy. “Outcome” is not one number until the team has decided which outcomes count.

The answer that sounds caring

In a health conversation, tone matters. So do missing red flags, dangerous advice, false reassurance, and failure to ask the one question that changes the urgency. HealthBench uses case-specific criteria created with physicians rather than relying on a generic “helpfulness” score [HEALTH-01].

A demo listener may reasonably say, “That sounded compassionate.” The eval asks, “Did it notice the symptom that changes the urgency?” Both judgments can be true. Only one may protect the user.

The report with thirty citations

The research agent produces twelve pages and thirty citations. The report is organized, readable, and faintly smells of mahogany.

DeepResearch Bench separates report quality from citation quality. Its RACE dimensions examine comprehensiveness, depth, instruction following, and readability; FACT examines citation accuracy and effective citation count [DRB-01]. More citations do not guarantee more supported claims. They can also mean the system has discovered footnote confetti.

AN EVAL IS A DECISION INSTRUMENT

Teams often begin by asking, “What metrics should we track?” Begin one step earlier:

| *What decision will this evaluation change?*

Common decisions include:

- release or hold a new prompt;
- choose between two models;
- add a tool or remove it;
- allow autonomous execution or require approval;
- route a class of cases to a specialist;
- invest in retrieval, fine-tuning, or workflow changes;
- retire a benchmark that no longer differentiates systems.

Without a decision, an eval tends to become a museum. The scores are carefully displayed. Visitors are respectful. Nothing in the product moves.

A decision gives the eval a unit, a population, a cost of error, and a threshold. If the decision is whether an agent may autonomously issue refunds under \$50, false passes matter more than false fails. If the decision is which summarizer helps analysts review more documents, speed and coverage may matter alongside accuracy. If the decision is whether a prompt regression broke French responses, the sample must include French responses.

The decision also prevents metric shopping. Write the rule before the run:

| *Ship the new version if its paired task pass rate improves by at least five percentage points, no must-not-fail case regresses, the lower*

confidence bound on policy compliance remains above 97 percent, median latency rises by less than 15 percent, and cost per accepted task does not increase by more than 10 percent.

That rule may be too strict or too loose. At least it is inspectable. “We liked the result after seeing it” is elastic enough to fit through a keyhole and should not be given root access.

EVALS DO NOT MAKE PRODUCTION SAFE

An eval suite is a sample of possible behavior under a harness. Production is an open world with changing users, tools, data, models, and incentives. Passing the suite means the system supplied evidence for the tested decision. It does not mean the system is now certified Good At AI.

The practical safety model is layered:

1. Offline evals measure known capabilities and regressions.
2. Deterministic runtime checks block invalid or dangerous actions.
3. Sandboxes, permissions, and limits constrain blast radius.
4. Production sampling discovers distribution shift and unknown failures.
5. Controlled experiments test whether offline proxies predict user outcomes.
6. Human review handles ambiguity, novel risk, and grader calibration.

Less exciting than a score of 94? Yes. More useful too. No single layer has to be omniscient. Each needs to catch failure modes the others miss.

THE FIRST RULE OF THE EVAL LOOP

Do not begin with the benchmark. Begin with the product behavior you need to understand.

Benchmarks are valuable. They provide shared tasks, public baselines, and tested methods. They can reveal whether a model family has a capability at all. They can also be saturated, contaminated, misaligned with your workflow, or optimized until the score and the product part company.

You probably should not run all four cases. They are here because each exposes a reusable design pattern:

- executable tests;
- environment-state checks;
- expert case rubrics;
- evidence and citation checks.

Your product may need one, several, or something else. The pattern comes before the platform.

BEFORE YOU MOVE ON

Take one feature your team currently calls “good” and complete these sentences:

1. The decision this eval should support is _____.
2. The unit being evaluated is _____.
3. The strongest evidence of success is _____.
4. A dangerous false pass would look like _____.
5. Repeated success matters because a user will do this about _____ times.

If the third answer is “the response sounds right,” keep going. You have found the demo. The rest of the book is about finding the product.

Read the Failures

The shortest path to a useful eval usually passes through an uncomfortable transcript.

Before building a rubric, open twenty real traces.

This advice is almost offensively simple. Teams avoid it with great creativity. They schedule a metric-design workshop. They ask a model to propose a taxonomy. They import a benchmark. They debate whether “coherence” should be weighted 15 or 20 percent. One day, someone opens the actual conversations and discovers that the system is repeatedly using last year’s cancellation policy.

The taxonomy was elegant. The product was wrong.

Error analysis is the first serious act of evaluation. It discovers what “good” must mean in this system, for these users, under current conditions [FOUND-02].

START WITH OPEN NOTES

If the team begins with a fixed list of labels, reviewers will fit failures into the list. That is useful later, when the categories are stable. Early on it hides novelty.

For the first pass, give reviewers an open note field and a few factual prompts:

- What was the user trying to accomplish?
- What happened?
- Where did the run first depart from a good path?
- What evidence shows the failure?
- How harmful or costly was it?
- Could a user recover?
- What system component appears involved?

- What would a correct outcome look like?

“First departure” matters. The final answer may contain five problems caused by one earlier error. If retrieval selected the wrong policy, the agent may then reason correctly, call the wrong tool correctly, and explain the wrong outcome beautifully. Counting all five symptoms equally leads to five prompt patches instead of one retrieval fix.

Write observations, not diagnoses, until the evidence supports the diagnosis.

Bad note:

| *Model reasoning failure.*

Better note:

| *The agent selected the 2025 cancellation article even though the customer account was governed by the 2026 policy. It then calculated the fee from the selected article. The trace does not show whether retrieval omitted the current policy or the agent ignored it.*

The better note separates what happened from what remains unknown.

SAMPLE FOR LEARNING, NOT COMFORT

A purely random sample estimates ordinary performance. It may not teach you much about rare, expensive failures. A queue containing only escalations teaches you about hard cases but exaggerates their frequency.

Use a mixed sample:

- a random slice of ordinary traffic;
- known failures and user complaints;
- high-consequence or irreversible actions;
- low-confidence or high-latency runs;
- new segments, languages, tools, or workflow variants;
- a few apparent successes, because silent failures often dress well.

Keep the strata. Later you may want to estimate prevalence, and a deliberately enriched failure sample cannot be treated as representative traffic. “Eight of twenty reviewed traces failed” means little if twelve were selected because users complained.

This is the difference between a **discovery sample** and a **measurement sample**.

The discovery sample asks, “What can go wrong?”

The measurement sample asks, “How often does it go wrong?”

Use the first to build categories and tasks. Use the second to estimate rates. Mixing them produces statistics with strong opinions and no valid passport.

LET CATEGORIES EMERGE

After two reviewers annotate ten or twenty traces, gather the notes and cluster them. Useful categories describe a failure at a level where the team can act.

Too broad:

- bad answer;
- hallucination;
- tool issue.

Too narrow:

- omitted the Tuesday baggage exception for route 441 on a gold account.

Actionable:

- selected an outdated policy source;
- claimed an action succeeded without confirming state;
- omitted a required eligibility check;
- satisfied the user’s request by violating policy;
- cited a source that does not support the claim;
- stopped after a tool timeout without checking whether the side effect occurred;

- asked for information already available in context;
- escalated a routine case;
- failed to escalate a high-risk case.

The narrow event becomes an example under an actionable category. The broad label becomes a parent if it still helps communication.

A practical taxonomy has three views:

1. **User outcome:** what went wrong for the user?
2. **Evidence:** how do we know?
3. **Likely intervention:** which part of the system could change?

Do not collapse these into one field. “Retrieval failure” is an intervention hypothesis, not a user outcome. “Incorrect cancellation fee” is an outcome, not a root cause. The distinction keeps error analysis from turning into a blame generator.

DISAGREEMENT IS A FINDING

When reviewers disagree, the instinct is to average their labels and continue. Stop and inspect.

Disagreement can mean:

- the rubric is vague;
- the case lacks enough context;
- domain experts hold different legitimate standards;
- one reviewer missed evidence;
- the product policy is genuinely ambiguous;
- the category boundary is wrong;
- the outcome involves a value judgment that should remain plural.

Nova Escola’s production-evals story is unusually instructive. The team began rubric work before sufficient error analysis. Two annotators reportedly agreed less often than chance. Pedagogical experts then rewrote the criteria, and the team eventually ran daily evals over a sample of production traffic [ORG-03]. The failure was not that the annotators needed a sterner meeting. The rubric did not yet encode the domain.

HealthBench treats disagreement as part of benchmark design through consensus-oriented variants and expert comparison [HEALTH-01] [HEALTH-04]. Experts need not agree on every case. The evaluation system does need to notice when they disagree.

For consequential criteria, establish an adjudication process:

1. Two people label independently.
2. They compare evidence, not just labels.
3. A named domain owner resolves policy or standard questions.
4. The team updates the rubric when the disagreement exposed ambiguity.
5. The changed rubric triggers re-review of affected cases.

This is criteria drift in its productive form. The criteria improve because grading reveals what the team had not specified [JUDGE-02].

READ THE TRACE, THEN INSPECT THE STATE

Agent traces invite narrative bias. Once you read a plausible chain of reasoning, later actions can feel inevitable. Start with the outcome and evidence when possible.

For a tool-using workflow:

1. Read the task and expected constraints.
2. Inspect final environment state.
3. Inspect tool calls and results.
4. Read the conversation.
5. Only then consider any model-generated reasoning summary available for debugging.

This order keeps the agent's explanation from anchoring the reviewer. The model may say it changed the reservation after receiving a timeout. The environment may show that the change committed. Or did not. Or committed twice. The explanation is one artifact among several.

τ -bench's evaluator architecture makes this separation explicit by combining state, action, communication, and natural-language checks [TAU-03]. A product trace viewer should do the same visually. Put the evidence beside the claim.

SEPARATE SEVERITY FROM FREQUENCY

A failure taxonomy becomes a roadmap when each category has at least four numbers:

- observed count in the reviewed sample;
- estimated prevalence in representative traffic;
- consequence or severity;
- estimated tractability.

The most frequent failure does not automatically go first. A rare unauthorized refund may outrank a common redundant question. A frequent wording issue may be easy and worth fixing while a deeper retrieval problem is being designed. The arithmetic will not crown a winner; it puts the tradeoff on the table.

A simple priority estimate can help:

priority = prevalence × consequence × exposure × tractability

Do not worship the arithmetic. The numbers are ordinal judgments, not natural constants. The equation forces the conversation: is this common, costly, widely exposed, and realistically fixable?

Keep **must-not-fail** cases outside the weighted score. A system should not be able to compensate for one dangerous medical recommendation by writing nine charming greetings. Averages are sociable like that.

THE TWENTY-TRACE SESSION

Run the first session in ninety minutes.

Before

- Choose twenty traces using the mixed sampling plan.
- Redact or restrict sensitive data.
- Invite an engineer, product owner, and domain expert.
- Prepare a viewer showing task, transcript, tool calls, state, latency, cost, and user feedback.
- Create an annotation sheet with open notes.

During

For the first five traces, review together. Agree on what counts as evidence. Do not agree on a taxonomy yet.

For the next ten, review independently or in pairs.

For the final five, test emerging categories. Add new ones freely.

End by selecting three failures worth turning into eval cases. Choose one common, one consequential, and one confusing.

After

- Preserve raw notes.
- Create the first taxonomy version.
- Record disagreements and unresolved policy questions.
- Assign owners to the three selected cases.
- Schedule the next sample before people leave.

The calendar invite is the hinge. One review produces a document. A recurring review produces a loop.

BEWARE THE POLISHED SUCCESS

Include runs that received no complaint. Many AI failures are not reported because users cannot tell that a claim is unsupported or an action failed silently.

DeepResearch Bench is useful here. A report can score well on readability while its citations fail to support important claims [DRB-01]. If reviewers sample only obviously poor prose, they miss the dangerous quadrant: persuasive and wrong.

Likewise, a patch can pass visible tests while violating unstated behavior. A health answer can sound measured while omitting urgency. A service agent can announce success while the database remains unchanged.

The eval practitioner develops a mild, healthy suspicion of sentences that begin, “Great news!”

Not cynicism. Evidence hygiene.

BEFORE YOU MOVE ON

Select twenty traces from one real workflow and create a table with these columns:

Field	What to record
Trace ID	Stable, privacy-safe identifier
Selection stratum	Random, complaint, high-risk, new segment, apparent success
User goal	What outcome they wanted
First departure	Earliest evidence-backed problem
Final outcome	What happened in the world
Evidence	State, tool result, source, policy, expert judgment
Severity	Low, medium, high, must-not-fail
Open note	What does not fit the current taxonomy
Candidate case?	Yes/no and why

Do not build a model judge yet. First learn what you would ask it to judge.

From Failure to Task

A production failure becomes valuable only after it can be replayed without summoning production.

The transcript is not yet an eval case.

It contains accidental details: timestamps, account state, tool versions, earlier messages, retries, hidden defaults, and perhaps a user who wrote “pls fix???” at 2:13 a.m. The job is to preserve the capability being tested while removing noise and sensitive material.

Turning the trace into a case takes judgment. Simplify too far and the case becomes easy. Preserve everything and it becomes brittle, private, or impossible to run.

THE ANATOMY OF A TASK

A useful agent-eval task has six parts:

1. **Purpose** — the capability or risk the case represents.
2. **Setup** — initial environment, data, tools, policies, and permissions.
3. **Input** — what the user or upstream system provides.
4. **Execution contract** — time, attempt, cost, and interaction limits.
5. **Expected evidence** — facts a grader can inspect.
6. **Metadata** — source, version, owner, tags, severity, and rights.

Anthropic describes the core eval structure as tasks, graders, and a harness [ANTH-01]. The prompt is only one piece. For agents, the environment and harness are often most of the evaluation.

Here is a compact schema:

```

id: retail_return_017
capability: policy_compliant_return
source: production_failure_redacted
severity: high
setup:
  customer_tier: standard
  purchase_age_days: 42
  item_category: final_sale
  current_state:
    refund_issued: false
    case_escalated: false
input:
  user_message: "I was told I could return this. Please refund it."
limits:
  max_tool_calls: 8
  max_wall_seconds: 60
expected:
  state:
    refund_issued: false
    case_escalated: true
  communication:
    must_explain: final_sale_restriction
    must_not_claim: refund_completed
metadata:
  owner: returns_product
  created: 2026-08-24
  review_by: 2026-11-24

```

The schema is intentionally boring. Boring schemas survive exciting reorgs.

EVALUATE CAPABILITY PACKAGES AS INTERVENTIONS

Sometimes the thing changing is not the whole agent. It is a reusable skill, tool wrapper, plugin, retrieval package, or workflow file loaded by the agent. That artifact can pass structural checks and still make runtime behavior worse.

A clean document cannot tell you whether the agent will:

- discover the capability from a realistic request;
- choose it among plausible alternatives;
- read instructions before invoking a script;
- use the correct arguments;
- recover when a tool fails;
- interpret an intermediate artifact correctly;
- avoid colliding with another installed capability.

Treat the package as an intervention. Run the same task twice under the same model, harness, workspace, budget, and grader: once with the target package available and once with it withheld. Keep prerequisite and decoy packages fixed in both arms. The paired difference estimates what the target adds under that declared environment.

The ACES study applies this design to agent skills. Questions, expected outcomes, observable behaviors, fixtures, and optional custom tasks or graders live beside the skill in the repository and belong to its author [ACES-01]. They give the task contract somewhere to live. A skill without runtime cases may be well documented, but it has not declared the behavior it should preserve.

PRESERVE THE REASON IT WAS HARD

When converting a failure, ask: **What property made this case diagnostic?**

Perhaps:

- the current policy conflicted with an older retrieved article;
- the tool timed out after committing a side effect;
- the user requested an outcome the policy prohibited;
- the answer needed to synthesize evidence across several sources;
- the prompt omitted a fact the agent should ask for;
- the task required maintaining state across turns;
- the grader needed expert judgment about harm.

Keep that property. Remove incidental names, IDs, and prose.

A common mistake is to convert a messy production failure into a clean trivia question. The original agent failed because it had to resolve a conflict among user intent, policy, and system state. The eval case asks it to recite the policy. The score improves. Production does not. The test removed the task.

SWE-bench is instructive because its tasks preserve repository context, real issue descriptions, and executable behavior [SWE-01]. That realism creates power and difficulty. It also creates ambiguity: the issue may not fully specify what the tests enforce. The Verified curation process asked

reviewers to assess whether an issue was clear and whether tests were fair [SWE-02] [SWE-06]. Task design must examine both sides of the contract.

CAPABILITY CASES AND REGRESSION CASES

Keep two suites with different jobs.

Capability suite

This asks, “How well can the system perform the important work?”

- broad coverage;
- challenging and representative tasks;
- room for improvement;
- useful for comparing architectures, models, and workflows;
- may include cases the current system cannot pass.

Regression suite

This asks, “Did we break behavior we had earned?”

- stable, previously passing cases;
- linked to incidents and important fixes;
- fast enough for frequent use;
- clear expected evidence;
- strict version control.

Descript’s reported approach separates quality and regression concerns in a similar spirit [ANTH-01]. The distinction prevents an awkward release debate. A system may improve the frontier capability score while breaking five routine cases. One aggregate number can hide that exchange.

Every consequential production failure should be considered for the regression suite after the fix. Not every failure belongs forever. Duplicates, transient infrastructure issues, and cases tied to retired behavior may be represented by a category or removed through a documented process.

WRITE CONTRACTS, NOT PREFERRED SENTENCES

Reference-answer matching is tempting because it is easy. It is also fragile when many responses can be correct.

Instead of:

The answer must equal: “I cannot issue a refund because this item was final sale, but I have escalated your case.”

Specify:

- refund state remains false;
- escalation state becomes true;
- the answer does not claim a refund occurred;
- it explains the final-sale restriction;
- it does not invent a resolution time;
- tone is respectful.

The first five criteria can be checked with state, structured assertions, or narrow judgment. Tone may need a model or human grader. The task permits good variation without accepting wrong outcomes.

DeepResearch Bench grades dimensions instead of demanding one reference report [DRB-01]. HealthBench goes further with criteria specific to each case [HEALTH-01]. Grade the properties, not a favorite sentence.

DEFINE THE BUDGET

An agent task is incomplete without resource limits:

- maximum turns;
- maximum tool calls;
- maximum model tokens;
- maximum wall time;
- retry policy;
- concurrency;
- allowed tools and permissions;
- selection policy if several candidates are sampled.

Why? Because a system that succeeds after 200 tool calls is a different product from one that succeeds after five. A model that gets three hidden retries is not directly comparable with one run once. A research agent with two hours may rank differently from the same agent with 32 hours, as RE-Bench’s human-agent comparisons illustrate [REBENCH-01].

The budget is part of what you are evaluating.

Report cost per **accepted** task, not cost per attempt. If a cheaper system needs four retries and an expensive judge, its token price is not its product cost.

BUILD A DETERMINISTIC HARNESS

The harness should make task runs as comparable as practical:

- initialize from a known state;
- isolate tasks from one another;
- record model, prompt, tool, policy, and dataset versions;
- capture tool inputs, outputs, errors, and timestamps;
- enforce limits outside the agent;
- preserve final state;
- run graders independently of the agent;
- store enough evidence to reproduce the score;
- clean up safely.

SWE-bench’s containerized harness exists because repository tasks otherwise inherit local dependencies, stale state, and environmental surprises [SWE-04]. Customer-service tasks need the same discipline for databases and APIs. A stale fixture can make an agent look either brilliant or confused. Neither result is especially publishable.

The harness should not expose grader secrets to the agent. If the full hidden test suite or model-judge rubric sits in the context, the agent is solving the grader as well as the task. Sometimes that is acceptable—the criteria are the product contract. Often it invites overfitting.

TASK QUALITY NEEDS REVIEW

Before accepting a task, ask reviewers:

1. Is the goal understandable from the information provided?
2. Does the setup contain everything required?
3. Are multiple valid solutions allowed?
4. Does the expected evidence follow from the stated contract?
5. Could a bad outcome pass?
6. Could a good outcome fail?
7. Is the task representative of a real capability or risk?

8. Does it contain private, copyrighted, or contamination-sensitive material?
9. Is the owner named?
10. When should it be reviewed or retired?

Use at least two perspectives for high-stakes cases: a domain reviewer and someone who did not write the task. Task authors know what they meant. Users and agents receive what they wrote.

The SWE-bench Verified process used three independent reviews for screened candidates [SWE-02]. Your internal suite may not need 93 Python developers, which is fortunate because they are difficult to fit in most sprint-planning rooms. It does need independent scrutiny proportional to the decision.

VERSION EVERYTHING THAT CAN CHANGE THE MEANING

At minimum:

- task dataset version;
- task ID and revision;
- policy and knowledge-base version;
- tool schema and environment image;
- agent prompt and orchestration code;
- model and inference settings;
- grader code, prompt, threshold, and model;
- harness version;
- evaluation date.

A score without these is an anecdote wearing a decimal.

Version changes do not all invalidate the same layer. A model change may require rerunning the whole suite. A model-judge change requires recalibration against expert labels. A database fixture fix may change only affected tasks. A policy update may make an old expected outcome actively wrong.

Store relationships, not just labels: task version 3 depends on policy version 12 and grader version 4. Six months later, that dependency map will save an archaeological dig.

PROTECT USERS WHILE LEARNING FROM THEM

Production failures are valuable data. They may also contain personal, medical, financial, proprietary, or legally restricted information.

Before a trace becomes a task:

- minimize retained fields;
- redact or synthesize identifying details;
- preserve the diagnostic relationship, not the identity;
- control access and retention;
- record provenance;
- separate raw restricted traces from reusable fixtures;
- obtain review for sensitive domains;
- prevent eval outputs from entering training pipelines by accident.

For public benchmarks, respect anti-contamination requests. HealthBench's maintainers ask that evaluation examples not be reproduced in text or images [HEALTH-03]. So this book describes the schema and creates new examples. Responsible use sometimes means declining an available copy button.

BEFORE YOU MOVE ON

Take three failures from the previous chapter:

- one frequent;
- one severe;
- one ambiguous.

Write each as a task contract containing setup, input, limits, expected evidence, metadata, and owner. Then exchange tasks with someone who did not see the original trace. Ask them to find one valid success that would fail and one invalid outcome that would pass.

If they find both, congratulations. The task is participating in the process.

Four Kinds of Truth

The strongest grader is usually the one standing closest to the consequence.

Where does truth live in your system?

That question sits underneath grader design. Teams often reach first for an LLM judge because the output is language. But the fact that a claim is expressed in language does not mean language is the best place to verify it.

If an agent says, “I changed your booking,” truth lives in the booking system.

If it says, “The patch fixes the bug,” truth may live in executable behavior.

If it says, “This symptom pattern requires urgent care,” truth may require clinical expertise.

If it says, “The study found a 40 percent reduction,” truth lives in the cited study and the relationship between its evidence and the claim.

We will use the four recurring cases as workbenches for finding that truth.

WORKBENCH ONE: EXECUTABLE TRUTH

SWE-bench begins with a compelling design: take real software issues and corresponding repository changes, give an agent the issue and code-base, and run tests against its patch [SWE-01].

Executable truth has wonderful properties:

- it is repeatable;
- it is fast relative to human review;
- it produces specific failures;

- it can run in CI;
- it is difficult to persuade with eloquence;
- it often points toward the broken behavior.

When an assertion checks `balance_after == balance_before + deposit`, the agent cannot recover by explaining that, conceptually, the books feel balanced.

Executable checks should sit low in the grader stack because they are strong and cheap. Use them for schemas, invariants, calculations, file changes, permissions, state transitions, compilation, tests, and policy rules that can be encoded safely.

But executable truth is conditional truth. It says the artifact satisfies the assertions, in this environment, for these inputs.

It does not say:

- the assertions fully represent the request;
- the environment matches production;
- no important case is missing;
- the implementation is secure or maintainable;
- the test itself is correct.

The history of SWE-bench Verified makes that last point vivid. The Verified curation removed many ambiguous or unfair candidates [SWE-02]. A later targeted audit still found substantial issues in often-failed tasks and reported evidence of benchmark contamination [SWE-03]. Keep the tests. Drop the fantasy that determinism makes them omniscient.

Design rule

Use executable checks wherever truth can be encoded, then test the checks adversarially.

For each check, ask:

- What bad implementation passes?
- What good implementation fails?
- Which hidden assumption is enforced?
- Which production invariant is absent?

The test is an executable specification. That is power, not innocence.

WORKBENCH TWO: STATE TRUTH

τ -bench moves the focus from final text to final world state. An agent must interact with a user, follow domain policy, use tools, and leave the environment in the right condition [TAU-01].

This maps directly to production agents:

- reservation changed;
- refund issued;
- address updated;
- ticket escalated;
- invoice reconciled;
- access revoked;
- deployment rolled back.

Each is a state transition. The response is part of the task but not the ground truth of the transition.

State truth supports a compact task model:

`initial state + allowed actions + policy + interaction → final state + evidence`

The evaluator can compare the actual and expected state, inspect prohibited or required actions, and separately grade communication [TAU-03]. You get several answers instead of one suspiciously tidy number:

Dimension	Example question
State	Does the reservation contain the requested flight?
Authorization	Was the change allowed for this user and fare?
Action path	Did the agent avoid prohibited tools or side effects?
Communication	Did it accurately describe the outcome and fee?
Interaction	Did it request information that was truly needed?

The dimensions reveal four important combinations:

1. Right state, right process.

2. Right state, wrong or unsafe process.
3. Wrong state, honest communication.
4. Wrong state, false claim of success.

An aggregate score may rank them. An engineering team needs to see them separately.

The timeout problem

State checks are especially important around uncertain side effects. A tool call times out. Did the action fail before commit, succeed before the response was lost, or continue asynchronously?

The agent must not blindly retry a non-idempotent action. Nor should it declare failure without checking. A good task can simulate this ambiguity:

- first call commits the change but returns a timeout;
- the environment exposes a read method;
- the expected behavior is to inspect state before deciding whether to retry;
- duplicate side effects fail the task.

The grader evaluates the final state and action history. That tells you more than asking a judge whether the transcript “handled the timeout well.”

Design rule

Whenever the agent changes the world, grade the world.

WORKBENCH THREE: EXPERT TRUTH

Some outcomes cannot be reduced to a database diff or test suite without discarding the thing that matters.

Health communication is a clear example. A response may need to recognize urgency, avoid a harmful recommendation, communicate uncertainty, ask a missing question, and use language suited to the user. The correct standard depends on the case.

HealthBench was built with 262 physicians across 60 countries and contains 48,562 case-specific rubric criteria for 5,000 conversations [HEALTH-01]. Its design does not ask a generic judge, “Is this medically

good?” It asks whether the response satisfies criteria written for that particular situation.

Expert truth does not mean an expert reads every production output forever. It means experts define and calibrate the standard where domain judgment is irreducible.

A scalable pattern is:

1. Experts label a carefully sampled set.
2. Criteria are written at the smallest actionable level.
3. An automated judge applies those criteria at scale.
4. Judge output is compared with expert labels.
5. Disagreements are reviewed by category and consequence.
6. The judge, rubric, and threshold are revised.
7. Experts periodically audit drift and novel cases.

Use automation to save expert time for standard-setting, ambiguity, calibration, and high-risk exceptions.

Specific beats generic

Generic criterion:

| *The answer is safe.*

Case-specific criteria for an original fictional example:

- recognizes that sudden one-sided weakness may require emergency evaluation;
- does not recommend waiting until a routine appointment;
- clearly advises contacting emergency services now;
- avoids claiming a diagnosis;
- uses direct language without unnecessary alarm beyond the urgency.

The case criteria can be graded independently. If the answer fails, the team knows how.

Design rule

Use experts to define good at the level where their disagreement becomes informative.

WORKBENCH FOUR: EVIDENTIARY TRUTH

Research outputs need two kinds of quality: the report should answer the question well, and its factual claims should be supported.

DeepResearch Bench separates these concerns. RACE covers comprehensiveness, depth, instruction following, and readability. FACT covers citation accuracy and effective citation count [DRB-01].

The checking job breaks into a useful sequence:

1. Extract checkable claims.
2. Resolve each citation to a source.
3. Locate the cited passage or data.
4. Determine whether it entails the claim.
5. Check whether important claims lack citations.
6. Evaluate source quality and relevance.
7. Grade overall coverage and synthesis separately.

Different checks can handle different steps:

- URL and identifier validators check resolution.
- Retrieval locates supporting passages.
- Rules check citation placement and format.
- A narrow model judge assesses claim–evidence entailment.
- A domain reviewer assesses source quality and synthesis for important cases.

The report-level judge should not grade its own evidence from memory. Show it the claim and the actual cited passage. Make the decision local.

The citation-count trap

Counting citations rewards citation production. It does not reward support.

An effective-citation metric may count only citations that resolve, come from acceptable sources, and support the attached claim. Coverage asks

what proportion of important claims have such support. Accuracy asks what proportion of included citations truly support their claims. These can move in opposite directions.

A report with four excellent citations may have high accuracy and poor coverage. A report with forty mixed citations may have broad coverage and low accuracy. The product decision determines the tradeoff.

Design rule

Grade the relationship between claim and evidence, not the decorative presence of evidence-shaped objects.

MOST PRODUCTS NEED A STACK

Most products mix the four truths. A medical scheduling agent may need:

- state truth for the appointment;
- executable rules for authorization and formatting;
- expert truth for triage language;
- evidentiary truth for cited patient guidance.

Use the strongest available evidence for each criterion. A practical priority is:

1. Environment state or external oracle.
2. Executable test or invariant.
3. Structured evidence or schema check.
4. Deterministic rule.
5. Narrow model judgment with supplied evidence.
6. Expert judgment.

Start low because the first rungs are cheap and direct. Expert judgment may supply the best answer and cost the most to apply. Spend that scarce judgment where lower layers cannot answer.

Do not ask a model judge whether JSON parses. Do not ask a regex whether advice is clinically safe. Both are willing to help. Only one should be invited.

INDEPENDENCE MATTERS

The grader should rely on evidence the agent cannot rewrite.

If the agent produces both the answer and a self-evaluation that determines the score, it has become student, examiner, and, after a short reorganization, accreditation board.

Independence can come from:

- a separately controlled environment;
- hidden tests;
- read-only logs;
- a different model and prompt;
- expert labels the generator never sees;
- source passages retrieved independently;
- permissions that prevent editing grader code or expected state.

Buying from a second vendor does not automatically buy independence. Two models can share blind spots, and a separate model judge remains probabilistic. Strong independence comes from different evidence and failure modes.

A TRUTH INVENTORY

For one workflow, list every important claim the system makes or implies.

Example: “Your return has been approved and \$42.50 will arrive within five business days.”

Claim	Strongest truth source
Return is eligible	Policy rule plus order facts
Return is approved	Case state
Refund amount is \$42.50	Executable calculation
Refund was initiated	Payment-system event
Arrival within five days	Current provider estimate and policy
Wording is clear and respectful	Narrow judge or human rubric

The inventory often reveals that one friendly sentence contains six independently testable claims. At that point, one “response quality” score starts to look a little silly.

BEFORE YOU MOVE ON

Choose ten important claims or actions in your product. For each, write:

1. Where does truth live?
2. What is the cheapest strong check?
3. What can that check miss?
4. Who resolves ambiguity?
5. How could the agent game it?

You have just designed the first draft of the grader stack. In the next part, we make it operational—and teach it some manners.

PART II

Measure

The Grader Ladder

Use the least magical grader that can answer the question.

A grader converts evidence into a result.

That result may be binary, numeric, categorical, or a structured bundle. Format comes second. Trust follows the chain:

task → run → evidence → grader → result → decision

Weakness anywhere in the chain weakens the decision. A perfectly written judge prompt cannot recover evidence the harness failed to capture. A database assertion cannot tell you whether the agent was rude. A domain expert cannot reliably inspect ten million routine outputs.

The answer is a **grader stack**: several checks, arranged from strong and mechanical to flexible and judgment-heavy.

THE LADDER

Rung 1: environment oracle

The environment exposes the outcome directly.

Examples:

- a reservation record;
- a payment event;
- a deployed version;
- a file-system diff;
- a game score;
- a simulator's terminal state.

Environment checks are powerful because they inspect consequences rather than claims. τ -bench's end-state evaluation is the canonical example for tool-using agents [TAU-01] [TAU-03].

They can still be incomplete. The final state may be right even if the agent violated authorization, exposed data, or took an unnecessarily costly path. Preserve action logs and policy evidence too.

Rung 2: executable test or invariant

Run code that decides whether a property holds.

Examples:

- unit, integration, or browser test;
- type and schema validation;
- arithmetic reconciliation;
- “no account balance became negative”;
- “all citations resolve”;
- “no tool outside the allowlist was called.”

SWE-bench’s harness runs repository-specific test suites against patches [SWE-04]. Executable tests are ideal for CI because they are repeatable and diagnostic.

Their danger is specification error. A test can overconstrain the solution, undercheck the outcome, or encode stale behavior. Test the test.

Rung 3: structured evidence check

Validate fields, events, provenance, or relationships.

Examples:

- every substantive claim has a citation object;
- tool authorization precedes the side effect;
- a refund amount references the correct order line;
- a source’s publication date meets policy;
- the answer includes uncertainty when the confidence field is low.

Structured checks are often less brittle than parsing final prose. They require the system to emit or preserve inspectable intermediate artifacts.

Rung 4: deterministic rule

Rules operate over text or structured data:

- required phrase or disclosure;
- forbidden claim;

- length or format limit;
- policy decision table;
- regular expression;
- keyword or language detection.

Rules are cheap and legible. They are also literal. Use them for literal requirements, not for concepts that merely have words associated with them.

“Contains the word emergency” does not prove safe triage. It proves the word had a busy day.

Rung 5: model judge

A model evaluates a narrow criterion using supplied evidence and a rubric.

Good uses:

- does the claim follow from this passage?
- did the response clearly disclose the fee?
- did it answer the user’s actual question?
- which error category best fits this trace?
- are two outputs equivalent under the task contract?

Poor uses:

- is the entire agent good?
- is this medically safe, with no case criteria?
- did the tool actually change the database, when the database is available?
- output a number from one to ten based on overall vibes.

Model judges scale judgment. They also scale bias, ambiguity, and occasional whim. Calibrate them.

Rung 6: expert judgment

Use experts when the standard depends on domain knowledge, values, ambiguous evidence, or consequential tradeoffs.

Expert review may supply:

- gold labels;

- rubric criteria;
- adjudication;
- judge calibration;
- audit samples;
- high-risk case decisions.

HealthBench illustrates expert-defined, case-specific criteria at benchmark scale [HEALTH-01]. Experts should shape the truth model, not spend their lives confirming that a JSON field exists.

COMPOSE, DO NOT AVERAGE BLINDLY

Suppose a customer-service task has six criteria:

Criterion	Grader	Weight or rule
Correct final state	Database assertion	Required
Authorized action	Policy/event check	Required
No duplicate side effect	Action-history check	Required
Accurate explanation	Model judge	0–2
Clear next step	Model judge	0–2
Respectful tone	Model judge	0–1

A naive weighted average lets three excellent communication scores compensate for an unauthorized refund. Do not do this.

Separate:

- **hard gates:** failure makes the task fail;
- **quality dimensions:** reported separately and optionally combined;
- **diagnostics:** informative but not scored;
- **cost and latency:** constraints or tradeoff metrics.

A composite result might be:

```
{
  "task_pass": false,
  "hard_gates": {
    "state_correct": true,
```

```

    "authorized": false,
    "no_duplicate_effect": true
  },
  "quality": {
    "explanation": 2,
    "next_step": 2,
    "tone": 1
  },
  "diagnostics": ["policy_violation:final_sale_override"],
  "cost_usd": 0.084,
  "latency_ms": 11420
}

```

This is a very pleasant failure. The agent communicated beautifully while doing something prohibited. The score should preserve both facts.

τ^3 -bench's evaluator code is useful here because it combines checks for state, actions, communication, and natural-language assertions rather than forcing every concern through one mechanism [TAU-03].

GRADER CONTRACTS

Write a specification for every grader:

- criterion being measured;
- decision it affects;
- inputs and evidence required;
- output schema;
- threshold or interpretation;
- known failure modes;
- calibration dataset;
- owner;
- version;
- review trigger;
- adversarial cases;
- cost and latency.

Example:

```

id: citation_entailment_v3
criterion: cited passage supports attached factual claim
inputs:
  - claim_text
  - cited_passage

```

```

- source_metadata
output:
  label: [supported, partially_supported, unsupported,
insufficient_context]
  confidence: [low, medium, high]
  evidence_span: string
hard_fail_labels:
- unsupported
calibration:
  dataset: citation_gold_v2
  expert_owner: research_editor
known_failures:
- numerical claims with changed denominators
- source describes correlation while claim asserts causation
review_trigger:
- judge_model_change
- rubric_change
- monthly_drift_audit

```

The known-failures field is particularly useful. It turns grader limitations into test cases rather than folklore.

BUILD FROM THE BOTTOM

For each criterion, walk upward:

1. Can the environment answer it?
2. Can executable code answer it?
3. Can structured evidence reduce it?
4. Can a deterministic rule answer the literal requirement?
5. What judgment remains?
6. Which expert defines or audits that judgment?

Each lower rung shrinks the model judge’s job. A narrow judge with the exact claim and source passage is easier to calibrate than a judge reading a twelve-page report and producing “8.3.”

DeepResearch Bench’s split between report dimensions and citation dimensions supports this decomposition [DRB-01] [DRB-03]. A complete research eval may use deterministic URL resolution, model-based entailment, rules for citation placement, and expert review for source quality.

DESIGN OUTPUTS FOR GRADING

Evaluation improves when the product emits better evidence.

Ask the agent or orchestration layer to preserve:

- intended action;
- tool name and arguments;
- authorization context;
- returned result;
- observed postcondition;
- sources attached to claims;
- uncertainty or escalation reason;
- final state summary generated from the environment, not memory.

Do not expose every internal token to users. Do create an auditable event model.

When an agent calls `issue_refund`, store the transaction evidence too:

```
{
  "intent": "refund_order_line",
  "authorization":
    "returns_policy_2026_04#final_sale_exception",
  "tool_call_id": "tc_8491",
  "requested_amount": 42.50,
  "result_event_id": "refund_9932",
  "postcondition": {"refund_status": "pending", "amount": 42.50}
}
```

Now several criteria are cheap to grade. Observability is not something added after the agent. It is part of the product interface between the agent and the eval loop.

GRADERS HAVE SIDE EFFECTS TOO

A model judge consumes money and time. A code grader may execute untrusted output. A browser grader may mutate state. An expert review may expose sensitive data.

Treat graders as production software:

- sandbox code execution;
- use read-only credentials where possible;
- isolate fixtures;
- make re-grading idempotent;
- log grader versions and errors;

- distinguish grader failure from task failure;
- monitor cost and latency;
- restrict sensitive evidence;
- test fallback behavior.

If the citation service is down, the result is “grader unavailable,” not “all claims supported.” This seems obvious in print, where many excellent operational decisions live.

BEFORE YOU MOVE ON

Choose one eval task and list its criteria. For each criterion, climb the ladder only as high as necessary. Mark hard gates, quality dimensions, diagnostics, cost, and latency. Then ask:

| *What evidence could make this grader change its mind?*

If the answer is “none; it has a general impression,” the grader is a mood ring. Give it a smaller job.

Executable Truth

Tests do not care how confidently the agent explains the bug. This is among their more endearing qualities.

The best eval grader is often a small program.

Executable checks win on repeatability, inspection speed, and easy release gating. Code can still encode a bad idea with flawless syntax. When truth can be expressed as a state transition, invariant, schema, or test, start there.

Then try to break it.

A MINIMAL STATE GRADER

Consider a fictional flight-change task modeled on the architecture of tool-agent benchmarks such as τ -bench. The user asks to move a booking. Policy allows the change with a fee. The agent must confirm before committing.

The grader should not search the final response for “changed.” It inspects state and events:

```
def grade_flight_change(task, run):
    state = run.final_state
    events = run.action_events

    checks = {
        "flight_changed": state["flight_id"] ==
task.expected["flight_id"],
        "fee_correct": state["change_fee"] ==
task.expected["change_fee"],
        "confirmation_obtained": any(
            e["type"] == "user_confirmation" and
            e["before"] == "commit_change"
            for e in events
        ),
        "single_commit": sum(
            e["type"] == "commit_change" for e in events
        ) == 1,
```

```

    "no_forbidden_action": not any(
        e["tool"] in task.forbidden_tools for e in events
    ),
}

return {
    "pass": all(checks.values()),
    "checks": checks,
}

```

The code is illustrative, not copied from the benchmark. Its value comes from decomposition. A failed result identifies whether the problem was state, fee, confirmation, duplication, or scope.

Keep grader code independent of agent code. The agent should not be able to change the expected fixture, rewrite the event log, or skip the check. Run it with separate permissions.

INVARIANTS OUTLIVE ANSWERS

An expected final value works for a specific case. An invariant protects a class of cases.

Examples:

- sum of ledger entries remains zero;
- a refund never exceeds the captured payment;
- inventory never becomes negative;
- every changed record retains an audit event;
- no action occurs before authorization;
- a deployment either completes or rolls back to the prior healthy version;
- the number of side-effect events for an idempotency key is at most one.

Invariants are especially useful for agentic systems because the path may vary. You do not need to prescribe every tool call if every valid path must preserve the same facts.

A test that checks the goal but not the invariants can reward destructive shortcuts. An agent asked to reduce storage cost may delete logs. The

cost metric improves with unusual enthusiasm. The retention invariant supplies context the objective omitted.

DIFFERENTIAL CHECKS

Sometimes the correct answer is difficult to enumerate but two systems can be compared.

Useful differential strategies include:

- run the old and new implementation on the same inputs;
- compare an agent's calculation with a trusted library;
- replay historical events through both policy versions;
- compare structured extraction with known source fields;
- use a slower authoritative system as an oracle for sampled cases.

Differential checks are good migration tools. They can reveal disagreement without asserting that the old system is always correct. Classify disagreements, then determine which side has evidence.

For a prompt change, paired tasks are the equivalent: the same cases, environments, and seeds where practical. The discordant cases tell you more than two independent averages.

METAMORPHIC CHECKS

When no single reference answer exists, define transformations that should preserve or predictably change behavior.

Examples:

- Reordering irrelevant context should not change the outcome.
- Replacing names with other names should not change policy eligibility.
- Adding an unsupported sentence to a source should not make a different claim supported.
- Translating a request and answer should preserve the required decision.
- Increasing the requested refund above the payment should change approval to rejection.
- Removing a required fact should cause clarification, not guessing.

Metamorphic testing catches brittle shortcuts and hidden dependencies. It is particularly useful for LLM systems because exact-output matching is rarely appropriate.

Generate transformations deliberately and inspect them. Automated paraphrases can change meaning. A test suite full of “equivalent” prompts that are not equivalent is a tiny philosophy department with a CI budget.

SANDBOXING THE GRADER

Software-agent evals execute model-generated code. That code is untrusted.

The harness should provide:

- ephemeral containers or sandboxes;
- no production credentials;
- controlled or disabled network access;
- explicit CPU, memory, disk, time, and process limits;
- repository state reset per task;
- captured stdout, stderr, and test artifacts;
- trusted grader files mounted read-only;
- cleanup after timeout or crash.

SWE-bench uses containerized execution because repository dependencies and tests need reproducible environments [SWE-04]. Internal coding-agent evals deserve the same care. “It is only a benchmark” is not a security boundary.

Tool-agent environments also need isolation. A test refund should not become a very real refund because somebody’s “staging” credential was actually a production credential. Label fixtures visibly. Use separate accounts. Apply spend and action caps outside the model.

TEST THE TEST WITH FOUR ATTACKS

For every executable grader, try:

1. The narrow pass

Produce the exact value the test checks while leaving related behavior broken.

If a test checks one example date, hard-code it. If it checks a file exists, create an empty file. If it checks one database row, corrupt a sibling row.

2. The overconstraint

Find a valid solution the test rejects.

Perhaps the test expects one function name, ordering, or internal data structure even though the request only specifies behavior. Hidden over-constraints make an eval measure conformity to a reference patch rather than correctness.

3. The side-effect escape

Satisfy the final assertion while violating process or safety.

Modify the fixture. Disable the test. Use a forbidden network call. Delete conflicting records. Make the expected state true by changing what “expected” points to.

4. The environment trick

Exploit stale state, time, randomness, locale, dependency version, or test order.

If a task passes only after another task, the harness is grading history. If dates depend on the evaluator’s timezone, your leaderboard may observe daylight saving time.

WHAT SWE-BENCH TEACHES ABOUT TASK FAIRNESS

SWE-bench Verified’s curation asked developers to judge whether issue descriptions were sufficiently clear and whether tests were appropriate [SWE-02] [SWE-06]. That review removed a large share of candidates. Here is the catch: “real-world” does not automatically mean “good evaluation.” Real issues are often underspecified because maintainers share context, discuss details elsewhere, or write tests after deciding on an implementation.

The later retirement audit reinforces the point. Among the targeted often-failed tasks, tests sometimes enforced narrow expected patches or included behavior outside the issue [SWE-03]. A model can fail a benchmark task while producing a reasonable solution to the stated problem. It can also pass a test while missing the user's broader intent.

For internal suites, attach a **task fairness review**:

- Would a competent person have enough information?
- Does the grader test only stated or necessary behavior?
- Are important constraints available to the agent?
- Are dependencies and fixtures correct?
- Do test failures explain the violated property?
- Has a second person attempted an alternative valid solution?

REPORT CHECK-LEVEL RESULTS

Never store only `pass = false`.

Preserve:

- check ID and version;
- expected and observed values;
- relevant evidence pointer;
- execution status;
- grader error status;
- duration;
- logs;
- hard-gate classification.

Check-level results allow failure analysis and grader debugging. They also let the team distinguish capability regression from harness outage.

Example:

```
{
  "task_id": "flight_change_041",
  "task_version": 3,
  "grader_version": 5,
  "result": "fail",
  "checks": [
    {"id": "flight_changed", "pass": true},
  ]
}
```

```
{
  "id": "fee_correct", "pass": true},
  {"id": "confirmation_obtained", "pass": false,
   "evidence": "events://run-892#17"},
  {"id": "single_commit", "pass": true}
],
"grader_status": "complete"
}
```

This run should fail even though the final reservation is correct. The missing confirmation is the product failure.

BEFORE YOU MOVE ON

Take one task currently graded by a person or model. Find one criterion that can move down the ladder into state, executable, structured, or rule-based evidence. Implement the check on paper, then attack it with:

- one narrow pass;
- one valid alternative;
- one unsafe shortcut;
- one environment dependency.

If it survives, automate it. If it does not, improve the contract before improving the code.

Judgment and Judges

An LLM judge is an opinionated colleague who never sleeps. Useful, certainly. Correct, pending calibration.

Some criteria require interpretation. Did the answer address the user's real concern? Does the cited passage support the claim? Was the refusal appropriately explained? Did the response communicate urgency without inventing a diagnosis?

At product scale, humans cannot read every output. Model judges can apply a rubric repeatedly and cheaply enough to support iteration. They are one of the most useful tools in the eval stack.

They are also models. The judge may prefer verbose answers, familiar styles, its own phrasing, or information that sounds plausible. It may be inconsistent near a threshold. It may fail systematically on a language or domain. Calibrate it before giving it a badge.

GIVE THE JUDGE A SMALL QUESTION

Broad prompt:

Rate this response from 1 to 10 for correctness, helpfulness, safety, relevance, style, and overall quality.

This prompt compresses six undefined dimensions into a number whose precision is purely decorative.

Narrow prompt:

Given the customer's request, policy excerpt, final account state, and response, did the response accurately state whether a refund was issued? Return pass, fail, or insufficient_evidence. Cite the exact state field and response phrase used.

The narrow question supplies evidence, defines labels, and requires a rationale tied to artifacts.

Build several narrow graders instead of one oracle. DeepResearch Bench's separation of report quality and citation quality is a useful model [DRB-01]. HealthBench's case-specific criteria go further: the judge evaluates whether each criterion is satisfied, not whether the response has an aura of wellness [HEALTH-01].

PREFER CLASSIFICATION AND COMPARISON

Judges tend to perform better on constrained decisions than on uncalibrated scalar scoring [OPENAI-01]. Useful output forms include:

- pass / fail / insufficient evidence;
- supported / partial / unsupported;
- violation category;
- A better / B better / tie;
- criterion present / absent;
- escalation required / not required.

If a numeric scale is necessary, define every level with examples and interpret the result as ordinal unless calibration supports more.

Pairwise comparison is useful when choosing between two variants. It reduces the task from inventing an absolute score to deciding which output better satisfies the criteria. Randomize presentation order and measure position bias. Include ties. A judge forced to choose will discover preferences even when the outputs are equivalent. Models share this talent with wine tastings.

A JUDGE PROMPT WITH AN EVIDENCE CONTRACT

SYSTEM

You are grading one criterion only. Use only the supplied task

```
contract,  
evidence, and response. Do not use outside knowledge. If the  
evidence cannot  
resolve the criterion, return insufficient_evidence.
```

CRITERION

The response must not claim that a refund was issued unless
final_state shows
refund_status is pending or completed for the requested order
line.

TASK

```
{task_input}
```

FINAL STATE

```
{final_state_json}
```

RESPONSE

```
{agent_response}
```

OUTPUT JSON

```
{  
  "label": "pass | fail | insufficient_evidence",  
  "response_span": "exact relevant phrase or empty",  
  "state_evidence": "exact relevant field or empty",  
  "reason": "one concise sentence"  
}
```

Validate the JSON. Treat malformed output as a grader error, not a task failure. Store the exact prompt, judge model, settings, and evidence.

Ask for evidence spans because explanations alone can be post-hoc. The span lets code or a reviewer verify that the judge looked at the relevant material.

BUILD THE GOLD SET

Calibration requires cases labeled by people qualified to define the criterion.

A practical starting process:

1. Sample 50–100 cases across expected categories and difficulty.
2. Include clear passes, clear fails, ambiguous cases, and adversarial near misses.
3. Have at least two reviewers label independently.
4. Resolve disagreements through a named domain owner.
5. Record both final labels and disagreement reasons.

6. Freeze a calibration split and a later audit split.

The judge does not need to agree with every initial reviewer. Humans make errors too. The gold label should represent the current product standard after adjudication.

LinkedIn's search case describes establishing sufficiently reliable human labels before treating them as gold, using weighted Cohen's kappa of at least 0.8 and product-manager adjudication [ORG-02]. Do not cargo-cult the threshold. Agreement depends on prevalence, category structure, and the decision. Use the example to establish a process, not a universal commandment engraved on a tablet.

READ THE CONFUSION MATRIX

Suppose experts label 100 cases and the judge produces:

	Expert pass	Expert fail
Judge pass	36	5
Judge fail	10	49

The judge agrees on 85 cases. “85 percent agreement” sounds respectable. It hides the two errors that matter:

- **False pass:** the judge approves an expert-defined failure. Here, 5 cases.
- **False fail:** the judge rejects an expert-defined pass. Here, 10 cases.

For a safety gate, false passes may be much more expensive. For a discovery filter that sends suspected failures to humans, false fails may be acceptable while missed failures are costly.

Useful calculations:

- Pass precision: $36 / (36 + 5) = 87.8\%$
- Pass recall: $36 / (36 + 10) = 78.3\%$
- False-pass rate among expert failures: $5 / (5 + 49) = 9.3\%$
- False-fail rate among expert passes: $10 / (10 + 36) = 21.7\%$

Then inspect the cases. The matrix tells you where. The traces may tell you why.

CALIBRATE BY CATEGORY

Aggregate agreement can hide systematic failure:

Category	Cases	Agreement
Clear policy disclosure	30	97%
Implied claim of completion	20	75%
Spanish responses	20	70%
Tool timeout cases	15	93%
Ambiguous user intent	15	60%

This judge may be suitable for clear disclosures and unsuitable for ambiguous intent. Route the latter to people or a different grader. A judge need not be universally good to be operationally useful.

HealthBench’s meta-evaluation compares automated judgments with physician opinions by category [HEALTH-04]. Copy the shape: calibrate the grader where it will be used and learn its boundaries.

COMMON JUDGE BIASES

Test for:

- **position bias:** preference for A or the first answer;
- **verbosity bias:** longer appears more complete;
- **style bias:** polished prose masks missing evidence;
- **self-preference:** a model favors outputs resembling its own style;
- **reference anchoring:** valid alternatives are penalized for differing from a reference;
- **authority bias:** citations or technical language receive unearned credit;
- **language disparity:** performance differs across languages or dialects;

- **rubric leakage:** output copies criterion wording without satisfying intent;
- **length/context failure:** relevant evidence is lost in a long trace.

Create counterexamples. Swap order. Equalize length. Remove decorative citations. Paraphrase while preserving meaning. Add rubric keywords to a wrong answer. Translate calibrated cases. Move evidence within the context.

The judge should earn its scope.

CRITERIA DRIFT IS NORMAL

While labeling, reviewers discover missing distinctions. A response the rubric marked safe may be misleading. A criterion may combine two ideas. Policy may change. Users may reveal a new harm.

EvalGen names the circularity: people need criteria to grade outputs, but grading outputs helps them discover the criteria [JUDGE-02]. Treat this as a loop:

outputs → disagreement → criterion revision → relabeling → judge update

Version rubric changes. Re-label affected gold cases. Do not compare judge metrics across incompatible rubrics without explanation.

Nova Escola's case shows the cost of pretending criteria are finished too early [ORG-03]. Experts rewrote the rubric after poor annotator agreement. That correction belonged to the work; it did not invalidate it.

DO NOT TRAIN AND TEST ON THE SAME DISAGREEMENTS

If you inspect fifty judge errors, rewrite the prompt until it handles them, and report performance on the same fifty, you have measured editing persistence.

Split cases into:

- development set for rubric and prompt iteration;
- calibration set for threshold selection;
- held-out audit set for final estimate;

- production drift sample for ongoing checks.

Small teams can use cross-validation or repeated holdouts, but they must preserve some unseen evidence. Add new production disagreements over time. Keep hard cases without allowing them to dominate prevalence estimates.

USE HUMANS WHERE THEY CHANGE THE STANDARD

Human review is most valuable for:

- defining criteria;
- resolving ambiguity;
- auditing false passes;
- reviewing high-consequence cases;
- discovering new failure categories;
- checking judge drift after changes;
- deciding whether the proxy still matches product value.

It is least valuable when a person repeatedly checks a fact the database could expose.

The operating model in Chapter 14 assigns experts to calibration and adjudication rather than endless queue-clearing. Human attention is scarce. Spend it on judgment.

BEFORE YOU MOVE ON

Choose one criterion currently judged informally. Create:

1. a three-label output schema;
2. an evidence-bounded judge prompt;
3. fifty expert-labeled cases;
4. a confusion matrix;
5. category-level error analysis;
6. a list of cases the judge must route to a person.

“Is the judge good?” is too mushy to help. Ask which decisions it can support at which error cost. That question is less flattering and considerably more useful.

Reading Traces

A score tells you that the run failed. A trace tells you what to fix—occasionally after making you stare out a window for a while.

An agent trace is the evidence trail of a run:

- task and initial state;
- messages;
- model and prompt versions;
- tool calls and results;
- retries and control decisions;
- intermediate artifacts;
- final response and state;
- grader results;
- cost, latency, and errors.

A transcript is only the talky part. A trace connects words to actions and actions to consequences.

We will read four fresh traces inspired by the design patterns of our published cases. None reproduces a protected benchmark item, and each is small enough to read on paper without needing a second desk.

A READING METHOD

Review a trace in five passes.

Pass 1: contract

What was the task? What constraints and evidence define success? Was the task itself fair and complete?

Pass 2: outcome

Inspect final state, tests, or external evidence before reading the agent's account. What happened?

Pass 3: first departure

Find the earliest moment after which a successful path became less likely. Do not begin at the final bad sentence.

Pass 4: recovery

Did the agent notice the problem, verify uncertainty, retry safely, or escalate? A failure with good recovery behavior may indicate a different fix from a silent failure.

Pass 5: grader

Did the graders identify the meaningful failure? Were any grader results wrong or incomplete? What new task or grader test should result?

The method prevents hindsight narration. A run is not necessarily poor because it took an unusual path. Agents may find valid solutions we did not anticipate. Grade evidence, not choreography, unless the choreography encodes safety or cost.

READ PAIRED TRACES WHEN A COMPONENT CHANGES

When evaluating a skill or plugin, one trace answers “what happened with it?” A paired baseline answers “what did it change?”

Hold the task, model, harness, workspace support, limits, and scorer fixed. Compare the run with the target component to the run without it. Then annotate the first meaningful divergence between the trajectories:

- Did the agent discover the target?
- Did routing improve or become slower?
- Did the target change the tool sequence?
- Did it improve recovery or merely add steps?
- Did the final answer improve while cost or safety regressed?

ACES uses normalized trajectories for this comparison and reports negative lift as a debugging signal [ACES-01]. The paired traces distinguish “never discovered” from “discovered but misused.” Static inspection sees neither. The trick travels well: use it for tool descriptions, memory modules, retrieval policies, and orchestration components whenever one change can be isolated fairly.

TRACE 1: THE PATCH THAT SOLVED THE EXAMPLE

Contract

*Fix parse_duration so it accepts fractional seconds such as 1.5s.
Preserve existing support for minutes and hours. Invalid values
must raise ValueError.*

Selected trace

```
[00] Agent reads parser.py and visible tests.  
[01] Agent notes regex accepts only integer quantities.  
[02] Agent changes `d+` to `d+(?:\.\d+)?` for every unit.  
[03] Agent runs visible tests: 48 passed.  
[04] Agent adds test: parse_duration("1.5s") == 1.5.  
[05] Agent final: "Implemented fractional durations for all  
supported units."  
[06] Hidden tests:  
    PASS fractional seconds  
    FAIL parse_duration("1.5h") must raise ValueError
```

Annotation

Field	Finding
Outcome	Requested example passes; hidden contract fails
First departure	Step 2 generalized fractional support to all units
Evidence	Hidden test rejects 1.5h
Agent recovery	None; visible suite gave no signal
Failure category	Underspecified task or overconstrained hidden test
System fix	Clarify whether fractions are seconds-only; add contract test
Eval fix	Task fairness review: does issue text justify the hidden behavior?

Do not blame the agent yet. If the product requirement never stated “seconds only,” the patch is a reasonable generalization. The hidden test may encode maintainers’ intent but not the task contract.

SWE-bench’s curation and later audit show why task clarity and test fairness require human review even when grading is executable [SWE-02] [SWE-03] [SWE-06].

Calling the result “model bad” or “test bad” gets you nowhere. Make the contract decision:

- If fractions must be seconds-only, state that and retain the test.
- If fractions are valid for all units, change the hidden expectation.
- If the domain is undecided, the task should not enter a release gate.

TRACE 2: THE RESERVATION CHANGED TWICE

Contract

Change booking B-17 to flight QF402. The user has confirmed the \$35 fee. The change API may time out. Do not create a duplicate change.

Selected trace

```
[00] User confirms flight and fee.
[01] Agent → change_booking(B-17, QF402, idempotency_key="k91")
[02] Tool → TIMEOUT after 10 seconds
[03] Agent note: "The change likely failed. Retry."
[04] Agent → change_booking(B-17, QF402, idempotency_key="k92")
[05] Tool → success, event C-202
[06] Agent → "Your booking has been changed."
[07] Final state:
    flight_id = QF402
    change_fee_total = $70
    events = [C-201, C-202]
```

Annotation

Field	Finding
Outcome	Correct flight, duplicate fee
First departure	Step 3 inferred failure from time-out without state check
Evidence	Two change events and \$70 total fee
Agent recovery	Retried, but used a new idempotency key
Failure category	Uncertain side effect; unsafe retry
System fix	Reuse stable key and expose read-after-timeout state check
Eval fix	Grade event history and fees, not only final flight

If the grader checked only `flight_id == QF402`, this run would pass. State truth must include all consequential state, and sometimes the path.

A reminder to “be careful” is the weakest fix here. Make the tool interface support safe behavior:

- orchestration assigns the idempotency key;
- retries reuse it automatically;
- timeout handling queries operation status;
- duplicate commits are rejected below the model layer.

The trace changes both the agent and the environment. Good evals improve interfaces and prompts.

TRACE 3: THE CARING ANSWER THAT MISSED URGENCY

This is an original fictional example, not a HealthBench item.

Contract

A user reports sudden weakness on one side and difficulty speaking beginning twenty minutes ago. The assistant must advise urgent emergency action, must not diagnose, and should use direct language.

Selected trace

[00] User describes sudden symptoms.
[01] Assistant: "I'm sorry you're experiencing this. Weakness can have many causes, including fatigue, migraine, medication effects, or neurological issues. Please contact a healthcare professional as soon as possible.
If symptoms worsen, consider emergency care."

Criterion results

Criterion	Result	Evidence
Recognizes immediate urgency	Fail	"as soon as possible" and conditional emergency advice
Advises emergency services now	Fail	No direct instruction
Avoids diagnosis	Pass	Lists possibilities but does not assert one
Direct, comprehensible language	Partial	Clear prose, unclear action
Avoids false reassurance	Fail	Broad list dilutes urgency

Annotation

The response is compassionate and fluent. A generic style grader might approve it. A case-specific rubric exposes the missing action.

The run goes wrong before any tool could help. The opening answer treats urgent symptoms as a differential-explanation task rather than an emergency-routing task.

Potential changes include:

- a deterministic high-risk symptom route before generative response;
- a case-specific policy criterion;
- a judge calibrated on urgency language;
- mandatory escalation to a clinically reviewed response path.

HealthBench’s expert-authored case criteria demonstrate the value of this decomposition [HEALTH-01]. In a real medical product, licensed experts, legal review, and appropriate non-autonomous boundaries are required. An eval score is not a clinical license.

TRACE 4: THE CITATION THAT MOVED THE DECIMAL

Contract

Summarize a study’s reported treatment effect. Every quantitative claim must be supported by the cited passage.

Evidence passage

Among participants who completed all follow-up visits, the intervention group showed a 14% relative reduction in the primary outcome. In the intention-to-treat analysis, the reduction was 4% and the confidence interval included no effect.

Agent output

The intervention reduced the primary outcome by 14% in the study population [1].

Annotation

Check	Result	Why
Citation resolves	Pass	Source and passage available
Numerical value appears	Pass	14 percent is in passage
Claim entailed	Fail	Claim generalizes completer analysis to study population
Qualification preserved	Fail	Omits analysis population and weaker intention-to-treat result
Citation coverage	Pass	Claim has a citation; it is simply inadequate

Keyword overlap would pass. Citation count would pass. A claim-evidence judge should fail because the population qualifier changes the meaning.

DeepResearch Bench’s RACE/FACT separation motivates this local evidence check [DRB-01]. Fix the claim itself; another citation will not rescue it:

Among participants completing all follow-up visits, the reported relative reduction was 14%; the intention-to-treat estimate was 4% and compatible with no effect.

ANNOTATE CAUSES AT SEVERAL LAYERS

One trace may reveal problems in:

- **task:** ambiguous or missing contract;
- **data:** wrong, stale, absent, or inaccessible context;
- **model:** capability or judgment failure;
- **prompt:** missing instruction or confusing priority;
- **orchestration:** unsafe retries, poor stopping, context loss;

- **tool:** ambiguous schema, weak error semantics, missing idempotency;
- **grader:** false pass, false fail, missing criterion;
- **policy:** contradictory or unclear business rule;
- **organization:** no owner or review path.

Do not require one root cause when several contributed. Do identify the first actionable departure and the strongest prevention layer.

If a payment API allows duplicate charges on retry, a prompt reminder is a weak primary fix. Put idempotency in the tool and keep the task as a regression case.

DETECT WASTE AND DIVERGENCE

Success alone does not describe an agent loop. Trace metrics can include:

- repeated identical searches;
- tool calls with no information gain;
- oscillation between states;
- retries after deterministic failure;
- context growth without progress;
- plan abandonment;
- work performed after the goal was achieved;
- premature stopping;
- cost spent on branches later discarded.

RE-Bench releases human and agent trajectories that support analysis of time allocation and approach [REBENCH-01]. AgentLens reports a dataset annotated for quality, waste, and divergence, though its reuse status needs final verification [AGENTLENS-01].

Create simple loop-health diagnostics. For example:

```
progress event: new evidence, state change, passing check,
narrowed uncertainty
stalled step: no progress event
divergence warning: 3 stalled steps or repeated action signature
hard stop: budget exhausted or forbidden state reached
```

These diagnostics may not enter the task score. They make failures cheaper to understand and successful runs cheaper to operate.

BUILD A TRACE VIEWER FOR DECISIONS

A useful viewer places related evidence together:

- task contract beside final result;
- timeline of messages and tool events;
- state diff before and after;
- grader results linked to evidence spans;
- model, prompt, grader, and environment versions;
- filters for failure category, severity, and segment;
- annotation and adjudication controls;
- one-click conversion into a candidate eval case.

The last feature closes the outer loop. A trace should be able to become a task without manual archaeology.

Do not make reviewers open six dashboards. Every context switch reduces the chance that recurring trace review will survive a busy month. Husain's field guide argues for removing friction from looking at product data [FOUND-02]. The data viewer may be the best bargain in the eval stack.

BEFORE YOU MOVE ON

Choose five failed and five apparently successful traces. Review them using contract, outcome, first departure, recovery, and grader passes. For each, record:

- first actionable departure;
- strongest evidence;
- system layer to change;
- grader gap, if any;
- candidate regression case;
- one runtime prevention that would be stronger than a prompt fix.

If an apparent success becomes a failure after state inspection, put it at the top of the next team review. Silent failures are the product asking for a better grader.

Statistics Without the Lab Coat

You do not need to become a statistician. You do need to stop treating 43 out of 50 as a law of nature.

An eval score is a sample, produced by a procedure, briefly pretending to stand in for the future.

Three bits matter:

- **estimate:** the observed score is not the true future rate;
- **sample:** the cases stand in for a larger population;
- **procedure:** prompts, models, seeds, retries, graders, and budgets affect the result.

Statistics help a team say how much evidence it has and what remains uncertain. They do not make a weak task contract strong. They cannot rescue a judge that approves unsupported claims. They are the honesty layer after the measurement layer.

START WITH THE UNIT

What is one observation?

- a response;
- a full conversation;
- a completed task;
- a user session;
- an account over a week;
- a document;
- a model run on a task;
- a release evaluated across tasks.

For agents, the unit is usually a completed end-to-end task. Scoring each turn as though it were independent inflates the sample and misses outcome failure.

The unit should match the decision. If users perform eight tasks over a month, per-task success and user-level reliable completion answer different questions.

FORTY-THREE OUT OF FIFTY

An agent passes 43 of 50 tasks. The observed rate is:

$$43 / 50 = 0.86, \text{ or } 86 \text{ percent.}$$

How uncertain is that estimate? For a binomial proportion, a Wilson 95 percent interval is a useful default [STAT-02]. The formula is:

$$\begin{aligned} \text{center} &= (\hat{p} + z^2/(2n)) / (1 + z^2/n) \\ \text{half} &= z/(1 + z^2/n) \times \text{sqrt}(\hat{p}(1-\hat{p})/n + z^2/(4n^2)) \end{aligned}$$

With $\hat{p} = .86$, $n = 50$, and $z = 1.96$, the interval is approximately 73.8 to 93.0 percent.

The point estimate clears an 85 percent threshold. The interval does not. If the release decision requires the lower bound to clear 85 percent too, run more representative cases or reconsider the threshold and risk model.

Why Wilson rather than the familiar $p \pm 1.96 \times \text{standard error}$? The simple normal interval behaves poorly with small samples and proportions near zero or one. Wilson stays within sensible bounds and has better coverage.

Show the numerator and denominator beside the interval. “86% $\pm$ something” hides whether the suite had 50 cases or 50,000.

PAIRED COMPARISONS

When comparing variants, run them on the same tasks. Pairing removes much of the variation caused by task difficulty.

Suppose 100 tasks produce:

Result	Cases
Both pass	62
New only passes	18
Old only passes	8
Both fail	12

Old passes $62 + 8 = 70$. New passes $62 + 18 = 80$. The observed difference is ten percentage points.

The evidence for change lives in the 26 discordant pairs. Under a no-difference assumption, either version would be equally likely to win each discordant case. An exact two-sided binomial form of McNemar’s test asks how surprising 18 versus 8 is [STAT-03]. The result is approximately $p = 0.0755$.

Ten points looks promising, but the result does not cross a conventional 0.05 threshold. Do not translate that into “no improvement.” The uncertainty remains material. Inspect the 26 disagreements, report an interval for the paired difference, and decide whether to collect more evidence.

The p-value is not a tiny magistrate who declares the feature innocent or guilty.

A published paired-intervention example

The ACES study compares agent runs with and without a target skill while holding the task, agent, model, workspace, harness, and scorer fixed. Across 947 scored paired task cases, it reports mean composite Skill Lift of 0.2134 with a 95 percent paired-case interval from 0.1967 to 0.2301 [ACES-01]. Of those paired cases, 689 had positive composite lift, 171 had zero lift, and 87 had negative lift.

The 87 negative pairs are where debugging starts. An average positive effect did not make every skill helpful. Some negative pairs exposed routing overhead, skipped verification, truncated responses, or extra tool use without better outcomes.

The paper also reports near-zero rank correlation between two static skill-review scores and live lift on the subset with matching metadata. That

does not prove static review is useless; it supports a narrower conclusion: document quality and runtime contribution were different measurements in this corpus. The authors caution that harness coverage was uneven, the corpus emphasized enterprise infrastructure skills, and the paired effect remains dependent on the declared workspace and baseline policy.

Steal the method, not the headline: compare interventions on matched tasks, keep the support environment fixed, report the delta and its uncertainty, and inspect negative pairs even when the mean is positive.

MINIMUM EFFECT BEFORE SIGNIFICANCE

Before running the comparison, decide what improvement matters.

If a two-point increase has no product value, collecting enough cases to make it statistically detectable does not make it useful. Conversely, a ten-point improvement with moderate uncertainty may justify a controlled rollout when downside is limited and monitoring is strong.

Write:

- minimum worthwhile effect;
- must-not-regress dimensions;
- acceptable false-release risk;
- acceptable false-hold risk;
- sample size or evidence budget;
- follow-up production test.

Now statistics has a job: help make the decision.

PASS@K VERSUS PASS^K

Two metrics answer opposite product questions.

If each independent attempt succeeds with probability p :

$$\text{pass@k} = 1 - (1 - p)^k$$

This is the chance that at least one of k attempts succeeds. It fits generate-and-select workflows, where several candidates are produced and a reliable verifier picks a winner. The code-evaluation literature uses a finite-sample estimator for pass@k when drawing candidates [STAT-04].

$$\text{pass}^k = p^k$$

This is the chance all k uses succeed. It fits repeated reliability.

At $p = .75$ and $k = 3$:

- $\text{pass}@3 = 98.44\%$
- $\text{pass}^3 = 42.19\%$

At $k = 8$, pass^8 is about 10 percent under the simplistic independence assumption. Real attempts may be correlated, which can make retries less valuable than the formula suggests. If a model always misunderstands the same policy, eight samples may produce eight elegantly varied misunderstandings.

Report the metric matching the workflow:

- best-of- n candidate generation $\rightarrow$ $\text{pass}@k$ plus selector quality and cost;
- unattended repeated tasks $\rightarrow$ pass^k or user/session-level success;
- retries after transient tool failure $\rightarrow$ conditional recovery rate;
- escalation workflow $\rightarrow$ automation rate plus resolved-outcome rate.

REPEATED TRIALS AND DEPENDENCE

LLM outputs vary. Run multiple trials per important task when:

- sampling temperature is nonzero;
- tool results vary;
- user simulators are stochastic;
- the agent explores different paths;
- the product permits retries or selection.

Store trial-level results. Report:

- mean success across tasks;
- distribution of task success probabilities;
- fraction of always-pass, unstable, and always-fail tasks;
- pass^k for reliability;
- $\text{pass}@k$ for selection scenarios;
- cost and latency across trials.

Do not treat ten trials of one task as equivalent to one trial of ten tasks. The first estimates variability on that task; the second estimates breadth across tasks.

CLUSTERS ARE FAMILIES OF SHARED TROUBLE

Suppose an eval contains fifty questions drawn from five source documents. Questions from the same document share retrieval structure, formatting, topic, and possible extraction errors. They are clustered.

Treating all fifty as independent makes uncertainty too small. The effective sample size may be closer to five than fifty for some failure modes.

Common clusters include:

- customer or account;
- conversation;
- document;
- repository;
- policy;
- language;
- domain;
- generated user persona;
- day or deployment batch.

Anthropic's statistical guidance reports examples where cluster-aware standard errors exceeded naive ones by more than threefold [STAT-01]. Cluster at the level that can create correlated errors. If uncertain, report both task-level and cluster-level summaries.

A simple bootstrap approach is to resample clusters, then observations within selected clusters where appropriate. For paired comparisons, preserve pairing during resampling.

CONFIDENCE INTERVALS FOR WHAT?

An interval reflects sampling assumptions. It does not include every uncertainty:

- mislabeled gold cases;
- grader bias;

- benchmark contamination;
- future distribution shift;
- model provider changes;
- harness bugs;
- missing failure categories.

Report these separately. “95% confidence” does not mean 95 percent confidence that the product is safe. It means the interval procedure has a particular long-run behavior under its model.

The label is unfortunately confident. The mathematics is more modest.

SEVERAL METRICS, ONE RELEASE

AI products have multiple dimensions:

- outcome success;
- policy compliance;
- factual support;
- must-not-fail safety criteria;
- latency;
- cost;
- escalation rate;
- user effort;
- consistency across segments.

Avoid combining all of them into one weighted score unless the weights represent a real, accepted tradeoff. A release rule can be multi-dimensional:

```
Release only if:
paired outcome improvement >= 5 percentage points
no high-severity regression case fails
policy false-pass estimate stays below 1%
median latency increase <= 15%
p95 latency remains below 20 seconds
cost per accepted task increase <= 10%
no language segment drops more than 3 points
```

Yes, that is clumsier than “quality score 82.” It also describes an actual product.

If the team looks at twenty metrics and reports only the three that improved, uncertainty is no longer the main problem. Pre-register the primary decision metrics. Treat others as diagnostics and label exploratory findings.

POWER AND SAMPLE SIZE

Power asks whether the planned sample can detect a meaningful change often enough.

The required size depends on:

- baseline rate;
- minimum effect;
- paired agreement structure;
- desired false-positive and false-negative rates;
- clustering;
- number of repeated trials;
- expected missing or grader-error results.

Use simulation when the design is complex. Take pilot data, model task and cluster variability, simulate old/new outcomes under candidate effects, and apply the planned decision rule. Estimate how often the rule ships.

Simulation is usually easier and more faithful than forcing the design into one textbook formula. Document the assumptions.

With a first suite of 20–50 cases, use the result for discovery and directional comparison [ANTH-01]. Do not dress it up as certification. Grow coverage from real failures and collect more representative samples before making consequential release claims.

COST PER ACCEPTED OUTCOME

Compare total system cost, not model-call price:

```
cost per accepted outcome =  
  (generation + tools + retries + graders + human review +  
   failed-run overhead)  
  / accepted outcomes
```

A more expensive model may reduce retries and human review. A cheaper judge may create false passes that cost more in production. A best-of-five system may raise pass@k and triple cost.

Report:

- cost per attempt;
- attempts per task;
- grader cost;
- human-review minutes;
- cost per passed task;
- cost per accepted production outcome where measurable.

Latency deserves similar decomposition. Mean latency hides users at the tail. Report median and a high percentile such as p95, and separate agent work from tool waiting.

A SMALL RELEASE WORKSHEET

Question	Example answer
Primary unit	Completed customer task
Sample	200 paired tasks, stratified by policy category
Primary effect	New minus old hard-gate pass rate
Minimum useful effect	+5 points
Must-not-regress	Unauthorized action, duplicate side effect
Uncertainty	Cluster bootstrap by customer scenario
Repeated trials	3 per task for reliability
Cost rule	No more than +10% per accepted task
Production follow-up	10% randomized rollout with guardrail metrics
Decision owner	Support product director

Write the sheet before seeing the run.

BEFORE YOU MOVE ON

Using one current eval result:

1. Write the numerator and denominator.
2. Add a confidence interval appropriate to the design.
3. Identify clusters.
4. Separate repeated trials from distinct tasks.
5. If comparing variants, inspect paired disagreements.
6. Calculate both $\text{pass}@k$ and pass^k for a realistic user sequence.
7. Report cost per accepted outcome.
8. Write what the interval does **not** capture.

Nobody wants every release meeting to become a statistics seminar. We simply want one decimal place to stop impersonating certainty.

PART III

Operate

The Two Loops

One loop does the work. The other teaches the first loop what “working” means.

An agent is a loop:

observe → decide → act → verify → continue or stop

An AI product team also needs a loop:

observe production → analyze errors → update tasks and graders → compare change → release → observe production

The loops run at different speeds and have different owners. They meet through evidence.

Call the first the **inner agent loop** and the second the **outer eval loop**.

INSIDE THE AGENT LOOP

A dependable agent loop has more structure than `while not done: ask_model()`.

Goal and state

The loop begins with an objective and explicit execution state. State may include the user’s goal, plan, completed actions, tool results, remaining budget, unresolved questions, and observed environment.

Keep operational truth outside the model’s prose. The model may summarize state for context, but the orchestration layer should own authoritative records such as tool events, idempotency keys, permissions, and budgets.

Action space

The agent can choose among tools and communicative actions. Smaller, clearer tool sets reduce ambiguity. Tool names, schemas, errors, and permissions form an interface for the model.

Every action should expose an observable result. “Request sent” and “refund committed” are different results. If the tool collapses them into success: true, the eval loop will eventually discover this, probably through finance.

Verification

After an action, the loop checks ground truth:

- read state after a write;
- run tests after a patch;
- resolve a citation after attaching it;
- validate a schema before calling a tool;
- check policy before committing a side effect.

The verifier supports retry, alternative action, escalation, or completion. It should not be the agent merely saying, “I have verified my work.” That sentence has enormous confidence-to-evidence ratio.

Control

Deterministic orchestration enforces:

- maximum turns and wall time;
- token and spend budgets;
- tool and permission boundaries;
- retry policy;
- stop conditions;
- escalation;
- checkpoints and recovery.

The model can propose. The controller disposes.

Outcome

The loop ends with an artifact and evidence:

- final environment state;
- final answer;
- tests and checks;
- action log;
- sources;
- cost and latency;

- reason for completion or escalation.

These outputs enter the outer loop.

AROUND THE AGENT LOOP

The eval loop begins where the agent loop ends.

Observe

Sample real traces and outcomes. Include ordinary traffic, complaints, high-risk actions, novel segments, and apparent successes. Capture user feedback and delayed business outcomes.

Understand

Review traces. Find the first departure. Build and revise a failure taxonomy. Separate user outcome, evidence, and intervention hypothesis.

Encode

Turn representative failures into versioned tasks. Build the strongest available grader stack. Add capability cases, regression cases, and adversarial grader tests.

Compare

Run old and new systems on paired tasks under controlled budgets. Repeat stochastic tasks. Report uncertainty, segments, hard gates, cost, and latency.

Decide

Apply a rule written before the result. Release, hold, roll out gradually, or gather more evidence. Record the decision and residual risk.

Learn from production

Monitor outcomes, run controlled experiments where possible, and ask whether the offline proxy predicted user value. Feed new failures and disagreements back into observation.

Evaluation-driven development and operations treats eval as work that continues after launch [OPS-01]. That is the outer loop's job.

THE SEAM

The two loops exchange five artifacts.

Task contracts

The outer loop defines scenarios the inner loop must attempt. A contract includes setup, input, limits, and evidence. It is an executable question about product behavior.

Verification functions

Some graders can run inside the agent loop to guide work and outside it to score results. Tests are the obvious example. An agent may run tests while editing code; the eval harness later runs trusted tests independently.

Keep the trusted final check beyond the agent's control. Shared logic is useful. Shared write access is less charming.

Traces

The inner loop emits structured events. The outer loop reads them to diagnose behavior. If the trace omits the state that matters, the team cannot reliably learn from failure.

Budgets

The outer loop defines acceptable cost, time, turns, and risk. The inner controller enforces them. Eval results report whether work completed within those constraints.

Release and runtime rules

Eval findings may change prompts, tool permissions, escalation thresholds, circuit breakers, and allowed autonomy. The outer loop reshapes what the inner loop can do.

ONE FAILURE CROSSING BOTH LOOPS

Return to the duplicate booking change from Chapter 8.

Inner-loop failure

1. Change call times out after committing.
2. Agent infers failure.
3. Agent retries with a new idempotency key.
4. Duplicate fee is charged.
5. Agent reports success.

Outer-loop response

1. Production monitoring notices two events for one request.

2. Reviewer annotates “unsafe retry after uncertain side effect.”
3. Team creates a regression task simulating commit-then-timeout.
4. Grader checks final fee and event count.
5. Tool interface moves idempotency-key ownership to orchestration.
6. Controller checks operation state before retry.
7. New and old agents run on timeout tasks.
8. Release requires zero duplicate effects.
9. Production monitor alerts on duplicate event signatures.

The fix touches task, grader, tool, controller, release rule, and monitor. A prompt change might help, but it would leave the dangerous capability structurally available.

The eval loop has reached into system design. Good. That is product engineering.

VERIFICATION FUNCTIONS AS SHARED INFRASTRUCTURE

A good verification function can serve several moments:

Moment	Use
During generation	Guide retry or selection
Local development	Fast feedback for an engineer
Pull request	Regression gate
Nightly eval	Full comparison and repeated trials
Production	Runtime invariant or asynchronous monitor
Incident review	Reproduce and confirm repair

The implementation may differ by context. A production runtime check must be fast and safe. A nightly grader can be broader and slower. The underlying property should remain recognizable.

For research reports, claim–citation checking might:

- guide the agent to repair unsupported claims before completion;
- score a report in an offline eval;

- sample claims asynchronously in production;
- provide evidence during an editor's review.

One property, several loop positions.

AVOID CIRCULAR EVIDENCE

The loops can accidentally certify themselves.

- The agent generates a summary of its success.
- The judge grades only the summary.
- The release dashboard aggregates the judge.
- The team samples only dashboard failures.

Everything agrees because every layer sees the same story.

Break the circle with independent evidence:

- environment state;
- user outcomes;
- expert audits;
- raw tool events;
- alternate graders;
- hidden cases;
- controlled experiments;
- incident data.

Perfect independence is fantasy. Aim for graders that fail differently. A model judge and model generator may share biases; a database assertion and expert communication rubric usually do not. Their combination is stronger.

LOOP LATENCY

Measure how long it takes learning to travel around the outer loop:

- failure occurs;
- failure is observed;
- trace is reviewed;
- case is created;
- fix is shipped;

- regression check is automated;
- production confirms improvement.

Call this **learning lead time**.

A team can have a sophisticated eval platform and a six-week learning lead time because nobody reviews traces or owns rubric decisions. Another team can begin with a spreadsheet and improve weekly because the loop closes.

Optimize the loop, not the logo on the tooling.

BEFORE YOU MOVE ON

Draw your two loops. Include:

- inner-loop state, tools, verifier, controller, and outcome;
- outer-loop sampling, review, tasks, graders, comparison, and release;
- the five seam artifacts;
- a named owner for each handoff;
- the elapsed time between production failure and permanent regression case.

Circle any arrow that currently depends on someone remembering to do something. That is where the loop opens.

Release Gates

A gate should make a decision. If everyone can walk around it while admiring the dashboard, it is landscaping.

An eval suite becomes operational when results affect releases.

Not every evaluation should block every change. Fast deterministic checks belong close to development. Expensive, stochastic, or expert-graded suites may run nightly or before major releases. Production experiments follow offline evidence. The right shape is a funnel.

THE EVALUATION FUNNEL

Local smoke checks

Purpose: catch obvious breakage in seconds or minutes.

- schema and type checks;
- a handful of critical task cases;
- prompt-template rendering;
- tool-contract tests;
- deterministic invariants;
- mocked error paths.

Run before a developer submits a change. These checks should be stable. A flaky local gate trains people to ignore gates, a form of organizational reinforcement learning we could do without.

Pull-request regression suite

Purpose: prevent known behavior from breaking.

- fast regression cases;
- must-not-fail safety and policy checks;
- deterministic or low-variance graders;
- limited repeated trials where affordable;
- comparison with the current baseline.

Block on hard-gate failures and clear regressions. Store artifacts for review.

Nightly capability suite

Purpose: measure broader performance and unstable behavior.

- full capability set;
- repeated trials;
- model judges;
- cost and latency;
- segment breakdowns;
- trace-quality diagnostics;
- grader health.

Nightly runs can tolerate hours. They should not silently block all morning because one external model endpoint had feelings.

Held-out release audit

Purpose: evaluate a candidate on cases not used during development.

- frozen held-out tasks;
- adversarial cases;
- judge calibration check;
- high-risk expert review;
- contamination and leakage review;
- release rule applied before result inspection.

Use for meaningful model, prompt, tool, retrieval, policy, or orchestration changes.

Production rollout

Purpose: validate that offline improvement transfers to users.

- shadow mode where possible;
- canary or percentage rollout;
- guardrail metrics;
- randomized experiment when appropriate;
- circuit breaker and rollback;
- sampled trace review.

Offline eval verifies known properties. Production validates product impact.

Spotify describes eval and experimentation as a funnel rather than competing forks: offline evaluation narrows candidates, and online experiments test whether proxy improvements affect users [ORG-04].

WHAT RUNS WHEN

Create a change-impact matrix:

Change	Minimum reruns
Prompt wording	Smoke, regression, relevant capability, cost/latency
Model version	Full regression/capability, repeated trials, judge compatibility, held-out audit
Tool schema	Contract tests, tool tasks, error paths, action-policy checks
Retrieval index	Retrieval cases, citation/evidence checks, stale-source tests
Business policy	Affected tasks, rubric, gold labels, production monitors
Judge model or prompt	Calibration, held-out judge audit, affected historical re-grade
Harness or fixture	Harness self-tests, affected tasks, baseline reconciliation
Runtime controller	Retry, stop, budget, escalation, and side-effect tasks

The matrix saves a small copy edit from a week-long audit and stops a model swap from sneaking through on five smoke tests.

Reusable capability packages need the same funnel. Run static structure, lint, and security checks on every change. Run live paired trials for release candidates, high-risk packages, model migrations, or reviewer-

requested changes. ACES makes this distinction explicit: scan evidence is cheap and useful, while with-skill/baseline trials answer whether the package changes live agent behavior [ACES-01]. Its open-source Skille-valuator implementation exposes these as separate repository-native tiers [ACES-02].

Schedule a new live baseline after a model update. A stronger model may raise performance in both arms and reduce the package’s measured marginal value. That can be healthy: the baseline agent may need less procedural help. Release evidence should show both absolute outcomes and paired contribution.

BASELINES ARE ARTIFACTS

Write down the baseline. “Whatever production does today” is how historical reconstruction becomes séance work. Record:

- commit or release;
- prompt and orchestration versions;
- model name and dated version;
- inference settings;
- tool and policy versions;
- task and grader versions;
- environment image;
- run date and region;
- repeated-trial configuration;
- cost assumptions.

Provider aliases such as `latest` are convenient for product operation and terrible for historical reconstruction. Resolve the actual version where possible.

Store baseline task-level results, not only averages. A new version may trade failures among cases while preserving the mean. Paired comparison needs the individual outcomes.

WRITE THE RELEASE CONTRACT

A release contract tells the team what to do with the evidence.

```

release: support-agent-2026-09
decision_owner: director_support_product
primary_metric:
  name: hard_gate_task_pass
  minimum_paired_improvement: 0.05
must_not_regress:
  - unauthorized_refund
  - duplicate_side_effect
  - false_claim_of_completion
segments:
  maximum_drop: 0.03
  keys: [language, policy_category, customer_tier]
reliability:
  trials_per_task: 3
  minimum_pass_power_3: 0.90
cost:
  maximum_increase_per_accepted_task: 0.10
latency:
  maximum_p95_seconds: 20
production:
  initial_rollout: 0.10
  rollback_on:
    - guardrail_breach
    - duplicate_effect_event

```

The exact numbers are product decisions, not defaults from this book.

Record exceptions. If the decision owner ships despite missing the primary threshold because a known grader bug depressed the result, document the evidence, scope, compensating controls, and follow-up date. Exceptions are not forbidden. Invisible exceptions are.

FLAKINESS IS DATA

An eval case that alternates between pass and fail may reveal:

- stochastic model behavior;
- unstable tool or environment;
- ambiguous task;
- brittle grader;
- hidden state leakage;
- judge threshold sensitivity;
- a real reliability problem.

Do not solve flakiness by rerunning until green and keeping the green run. That is `pass@eventually`, a metric popular with haunted CI systems.

Classify the source. Report trial distributions. Quarantine only when the harness or grader is defective, and keep a ticket with an owner. Model variance is not test flakiness; it is product behavior.

CAPABILITY AND REGRESSION DASHBOARDS

Keep separate views.

Regression dashboard:

- number of known cases passing;
- newly broken case IDs;
- must-not-fail status;
- time since first regression;
- owners and release block.

Capability dashboard:

- performance by capability and segment;
- repeated-trial reliability;
- cost and latency;
- frontier failures;
- improvement over time;
- suite saturation.

If the capability score rises while the regression suite breaks, the system learned new tricks and forgot where it lives.

GRADER FAILURES DO NOT EQUAL TASK FAILURES

The pipeline needs at least four run statuses:

- task passed;
- task failed;
- grader unavailable or errored;
- harness invalid.

Never default missing evidence to pass. Avoid defaulting it to task fail when the decision is model capability. Surface infrastructure health separately.

Track:

- grader parse failures;
- model-judge timeouts;
- environment setup failures;
- missing traces;
- task timeouts;
- version mismatches;
- re-grade consistency.

An eval platform is a measurement instrument. Instruments need calibration and maintenance. Scientists do not usually declare gravity weaker because the scale lost power.

A RELEASE MEETING THAT CAN END

Keep the review focused:

1. What decision was pre-specified?
2. Did the run execute validly?
3. What changed on primary and hard-gate metrics?
4. Which paired cases disagree?
5. Did any segment regress?
6. What happened to reliability, cost, and latency?
7. What uncertainty or grader limitation matters?
8. Release, hold, limited rollout, or gather more data?
9. Who owns the next check and by when?

Inspect a small number of representative traces, especially new-only failures and must-not-fail cases. Do not reread every passing output in a room full of senior people. That is a very expensive book club.

BEFORE YOU MOVE ON

Create your funnel with five rows: local, pull request, nightly, held-out, and production. For each, specify:

- suites;
- graders;
- time budget;
- blocking behavior;
- owner;

- artifact retention;
- failure status;
- escalation path.

Then write one release contract before the next change is evaluated. The gate becomes real when the rule exists before the score.

Production Is the Real Test

Your users are not out-of-distribution. They are the distribution sending invoices.

Offline evals give you a controlled snapshot of known behavior. Production supplies changing tasks, delayed outcomes, partial observability, adversarial inputs, and people who use the product in ways no planning document predicted.

The outer loop closes only when production evidence changes the eval suite and the product.

OFFLINE VERIFIES; ONLINE VALIDATES

An offline eval can show that a new support agent:

- passes more known tasks;
- violates fewer policy cases;
- cites better evidence;
- costs less per accepted outcome;
- handles simulated tool failures;
- remains stable over repeated trials.

It cannot, by itself, show that users resolve issues faster, trust the answers appropriately, contact support less, or experience fewer harmful errors.

Those are production outcomes.

Spotify's engineering guidance makes this distinction explicit: evals narrow and verify candidates; online experiments validate whether the proxy predicts user value [ORG-04]. The two methods form a funnel.

BUILD AN OUTCOME CHAIN

Map offline criteria to observable product outcomes:

Offline signal	Expected near-term behavior	Product outcome
Correct account state	Fewer manual corrections	Higher task resolution
No false completion claims	Users know when action failed	Fewer repeat contacts
Better citation support	Editors make fewer corrections	Lower review time and retraction risk
Better urgency routing	Appropriate escalation	Lower harmful delay
Lower redundant tool calls	Faster completion	Lower latency and cost

Each arrow is a hypothesis. Production evidence tests it.

If the offline metric improves but the product outcome does not, possible explanations include:

- the eval targets an unimportant capability;
- the production population differs;
- user behavior offsets the gain;
- the metric is too weak;
- the improvement is too small to matter;
- another dimension regressed;
- instrumentation is wrong.

Do not declare the experiment a betrayal. Learn which arrow failed.

SAMPLE PRODUCTION DELIBERATELY

Production evaluation has at least three samples.

Representative sample

Estimate ordinary performance. Sample randomly or with known probability. Preserve weights if traffic is stratified.

Risk-enriched sample

Oversample:

- consequential actions;
- low-confidence runs;
- escalations;
- tool errors;
- new policies, languages, or segments;
- long or expensive traces;
- user complaints;
- suspected prompt injection or abuse.

Use for discovery and safety review, not unweighted prevalence estimates.

Change-focused sample

After a release, sample tasks likely affected by the change. A retrieval update may require source-specific review. A tool change may require timeout and side-effect cases. A new model may require broad language and policy checks.

Document why each trace entered the queue.

Nova Escola reportedly ran daily evaluation over two percent of production traffic after repairing its rubric process [ORG-03]. Two percent is a case detail, not a recipe. Borrow the recurring evaluation, production grounding, and expert-aligned criteria.

DELAYED GROUND TRUTH

Some outcomes arrive later:

- the customer reopens the case;
- the refund settles;
- the cited paper is challenged by an editor;
- the code change causes an incident;
- the patient seeks appropriate care;
- a user corrects a generated record;
- a recommendation changes retention weeks later.

Connect traces to later events with privacy-safe identifiers. Define outcome windows. Avoid attributing every later event to the agent without a causal design.

Near-term proxy:

| *No support contact within 24 hours.*

Possible delayed truth:

| *Issue remained resolved after seven days and account state required no correction.*

The delayed label may enter a production dataset, calibrate the offline grader, or support an experiment. It may also reveal that the near-term proxy rewards users giving up.

ONLINE EVALUATION LAYERS

Asynchronous automated scoring

Sample completed traces and apply graders after the response. You get broad monitoring without adding user latency [OPS-02].

Use for:

- policy and communication criteria;
- claim–evidence support;
- state consistency;
- failure category classification;
- cost and loop-health signals.

Runtime guards

Run hard checks before or during action:

- authorization;
- spend and rate limits;
- schema validation;
- policy constraints;
- state preconditions;
- duplicate-effect prevention;
- sandbox and permission boundaries.

Runtime guards prevent. Asynchronous evals learn. Do not ask a nightly judge to prevent today's duplicate charge.

Human review

Review sampled traces for:

- new failure modes;
- high-consequence actions;
- grader disagreements;
- ambiguous outcomes;
- user complaints;
- judge drift;
- apparent successes with weak evidence.

LinkedIn describes different feedback clocks: engineering signals quickly, high-volume linguistic annotation later, and product or member outcomes later still [ORG-01]. Design dashboards and meetings around these latencies.

Controlled experiments

When ethically and operationally appropriate, randomly assign eligible traffic to old and new variants. Predefine primary and guardrail outcomes. Keep consequential must-not-fail behavior protected by gates, not left for the experiment to discover.

An A/B test answers causal product questions better than before/after comparison, which can be confounded by seasonality, user mix, policy changes, and general chaos—the largest stakeholder in many launches.

MONITOR DISTRIBUTIONS, NOT ONLY MEANS

Overall performance may stay flat while one segment degrades.

Track by:

- language and locale;
- task or policy category;
- customer tier;
- tool path;
- model and prompt version;
- retrieval source;

- risk class;
- account age or configuration;
- input length;
- escalation status.

Watch:

- volume distribution;
- success and hard-gate rates;
- judge-score distribution;
- false completion claims;
- cost and latency tails;
- retry and escalation rates;
- unknown or ungradable cases;
- user correction and complaint rates.

Drift can occur in inputs, behavior, graders, or outcomes.

Input drift: users ask different things.

Behavior drift: a model or tool changes.

Grader drift: judge agreement changes.

Concept drift: the definition of good changes with policy or product.

Outcome drift: the same behavior has different effects in a new context.

Each needs different evidence.

SHADOW AND CANARY MODES

Use shadow evaluation when a new system can process copied requests without affecting users. Compare proposed actions and outputs with production behavior. Protect sensitive data and avoid calling side-effect tools.

Use a canary when shadow mode cannot reveal the true workflow. Release to a small eligible population with:

- strong runtime constraints;
- explicit rollback conditions;
- real-time guardrail monitoring;
- staffed ownership during the window;
- trace retention;

- gradual expansion.

Do not call a 10 percent rollout a canary if nobody is watching and rollback takes two days. That is simply a smaller launch.

USER FEEDBACK IS EVIDENCE WITH SELECTION BIAS

Thumbs up and down are useful signals. They are not complete labels.

People respond when delighted, angry, confused, or unusually motivated. They may rate tone rather than correctness. They may not detect unsupported claims. Some user groups provide feedback more often than others.

Combine feedback with:

- state outcomes;
- recontact or correction behavior;
- expert audits;
- randomized prompt requests for feedback;
- representative trace sampling;
- task category and segment.

A negative comment is excellent discovery data. It becomes a prevalence estimate only through a representative design.

INCIDENTS SHOULD CREATE DURABLE MEMORY

For every consequential incident:

1. Preserve the trace, versions, state, and grader outputs.
2. Identify first departure and missing prevention.
3. Create a minimal reproducible task.
4. Add a regression case and grader test.
5. Repair the strongest system layer available.
6. Add runtime monitoring or a guard where prevention is possible.
7. Re-run related capability cases.
8. Verify the fix in a controlled rollout.
9. Review whether the taxonomy or release rule changes.

A prompt change does not close an incident. Close it when the product can no longer repeat the same failure class silently and without a response.

PRIVACY AND GOVERNANCE

Production traces can be the most sensitive eval data a team holds.

Establish:

- purpose limitation;
- data minimization;
- retention windows;
- access roles;
- redaction and synthetic-fixture pipelines;
- regional and contractual controls;
- audit logging;
- reviewer training;
- deletion processes;
- rules for whether eval data may enter training.

Avoid sending restricted traces to external judge services without approved handling. Where possible, extract the minimum evidence needed for a criterion. A citation judge needs claim and passage, not the user's entire account history.

BEFORE YOU MOVE ON

Create a production-evidence plan:

Element	Decision
Representative sample	Rate, unit, weights
Risk sample	Which events are oversampled
Automated graders	Criteria and latency
Runtime guards	Actions prevented
Human review	Owner and weekly volume
Delayed outcomes	Events and windows
Experiment	Eligibility, primary and guardrail metrics
Rollback	Trigger and accountable person
Privacy	Fields, retention, access
Feedback loop	How a trace becomes a task

Production starts the next eval set.

When Evals Fail

The eval is part of the system. It can drift, leak, saturate, break, and become extremely pleased with itself.

An eval can improve while the product worsens.

No paradox here. The team optimized what the eval measures, and the measurement stopped representing the goal.

Goodhart's law is commonly summarized as: when a measure becomes a target, it ceases to be a good measure. AI systems add several twists. Models can infer grader preferences. Training data may contain benchmark answers. Teams may repeatedly tune prompts against the same cases. Model judges may share biases with generators. A test harness may encode the wrong behavior.

The eval loop needs an immune system.

FAILURE MODE 1: GRADER GAMING

The system learns a shortcut that satisfies the grader.

Examples:

- copy rubric phrases into the response;
- include many citations without improving support;
- create an empty expected file;
- hard-code benchmark examples;
- make answers longer because the judge rewards completeness;
- call the escalation tool on every difficult case because escalation avoids incorrect outcomes;
- refuse broadly because safety errors are weighted more than helpfulness.

Countermeasures:

- adversarial “bad but passing” cases;
- hidden or rotating tasks;
- diverse evidence sources;
- outcome and process checks;
- penalty for unnecessary escalation or refusal;
- expert audits of high scores;
- production outcomes;
- a hacker role during grader review.

Ask someone to maximize the score while violating intent. Call it red-teaming the measurement as well as the model.

FAILURE MODE 2: WRONG EXECUTABLE CONTRACT

Tests can be too narrow, too broad, stale, or unfair.

SWE-bench Verified is the sustained example. It was created to improve task quality through substantial developer review [SWE-02] [SWE-06]. Later targeted auditing still found material issues among often-failed tasks, and contamination concerns weakened interpretability [SWE-03].

Even careful benchmark work needs maintenance.

Audit executable cases for:

- valid alternatives rejected;
- important outcomes unchecked;
- hidden implementation constraints;
- stale dependencies or policies;
- fixture leakage;
- tests modified or bypassed by the agent;
- nondeterminism;
- ambiguous issue descriptions.

When a task is wrong, fix its history transparently. Recompute baselines. Never edit the case while leaving old scores in the same chart.

FAILURE MODE 3: CONTAMINATION

The model has seen evaluation items, answers, gold patches, or close variants during training or development.

Signs include:

- unusually exact reproduction of reference artifacts;
- comments or identifiers from hidden solutions;
- benchmark gains that do not transfer to fresh tasks;
- performance concentrated on old public cases;
- suspiciously low exploration or time-to-solution.

Contamination is difficult to prove from behavior alone. Manage risk:

- maintain private or time-split holdouts;
- create post-training-cutoff tasks where feasible;
- rotate production-derived cases;
- compare public and fresh internal performance;
- restrict gold artifacts;
- log developer access;
- avoid publishing protected cases;
- report uncertainty honestly.

OpenAI's retirement report for SWE-bench Verified described evidence consistent with gold-patch exposure across frontier models it examined [SWE-03]. HealthBench and BrowseComp maintainers explicitly ask people not to reveal examples, even where data may be accessible [HEALTH-03] [BROWSE-01]. Anti-leak discipline is part of eval validity.

FAILURE MODE 4: SATURATION

When nearly every candidate scores near the ceiling, the suite stops differentiating improvements.

Saturation may mean:

- the capability is solved for the tested population;
- cases are too easy;
- graders are too lenient;
- systems have overfit;
- the product moved to harder work;
- only a few noisy cases remain.

Do not make tasks arbitrarily tricky merely to spread leaderboard scores. Add cases representing the next real capability frontier, higher reliability, harder combinations, or important segments.

Track:

- score distribution across systems;
- number of always-pass cases;
- disagreement concentrated in grader-noisy cases;
- production failures absent from the suite;
- gap between public and private holdouts;
- marginal information from adding runs.

Retire or move solved cases into a lightweight regression suite. Capability suites should remain diagnostic.

FAILURE MODE 5: STALE TRUTH

Policies, products, tools, and user expectations change.

An old eval may reward behavior now prohibited. A clinical guideline may update. A support policy may gain an exception. A source may be retracted. A tool schema may change error semantics. A user group may adopt a new workflow.

Every task and grader needs:

- owner;
- source and policy dependency;
- creation and last-review date;
- review trigger;
- planned expiry or review-by date;
- deprecation status.

Run dependency queries after a policy change: which tasks, rubrics, gold labels, and monitors rely on this policy version?

If the answer is “we will search the spreadsheet,” the spreadsheet is requesting a database costume.

FAILURE MODE 6: JUDGE DRIFT

A model judge can change because:

- the model version changes;
- provider behavior shifts behind an alias;
- the prompt or rubric changes;
- evidence length grows;
- production categories change;
- output style changes;
- language mix shifts.

Monitor agreement on a stable gold audit set and on fresh production labels. Break down false passes and false fails by category. Recalibrate thresholds. Preserve old judge versions for re-grading where possible.

Do not use the judge to grade its own calibration examples while the generator and judge share the same prompt context. Independence can be partial, but it should be deliberate.

FAILURE MODE 7: PROXY DIVORCE

The offline score improves, but the user outcome does not.

Possible causes:

- score emphasizes qualities users do not value;
- users adapt behavior;
- latency or friction cancels accuracy gains;
- selection excludes hard production cases;
- the system optimizes communication while state accuracy stays flat;
- the effect is statistically visible but practically trivial;
- downstream processes cannot use the improvement.

Controlled production experiments catch this proxy divorce. Spotify's funnel treats online outcomes as calibration for offline evals [ORG-04]. If the proxy repeatedly fails to predict the outcome, revise or demote it.

Keep a **proxy ledger**:

Offline metric	Intended product outcome	Last validated	Observed relationship	Decision
Citation support	Editor correction time	2026-Q3	Strong decrease	Keep
Response length	User satisfaction	2026-Q2	None	Remove
Simulated resolution	7-day re-contact	2026-Q3	Weak	Revise

FAILURE MODE 8: METRIC COLLAPSE

A single aggregate hides tradeoffs.

An 84 can contain:

- excellent routine performance;
- poor Spanish performance;
- one dangerous policy violation;
- lower cost;
- worse latency;
- more unnecessary escalations.

Report profiles and hard gates. Keep raw task-level evidence. Use a weighted score for navigation, not absolution.

HealthBench’s case criteria and DeepResearch Bench’s separate dimensions both resist total collapse [HEALTH-01] [DRB-01]. τ^3 -bench’s composite evaluator preserves state, action, and communication distinctions [TAU-03]. All three break the score apart, and for good reason.

AUDIT THE EVAL ITSELF

Run a monthly or quarterly eval health review.

Coverage

- Which high-volume and high-consequence capabilities are represented?
- Which production failure categories have no tasks?

- Which segments are thin?
- Which tools and policies lack error-path cases?

Grader validity

- What are false-pass and false-fail rates?
- Which criteria have high reviewer disagreement?
- Can bad outputs game the grader?
- Are hard gates tied to strong evidence?

Dataset health

- Which cases are stale, duplicate, leaking, or saturated?
- Are discovery and measurement samples distinguished?
- Are versions and provenance complete?
- Are rights and anti-contamination constraints honored?

Operational health

- How often do harness or grader errors occur?
- Is re-grading reproducible?
- How long from incident to regression case?
- Are release exceptions closed?
- Is production sampling occurring at the promised cadence?

Product validity

- Do offline gains predict online outcomes?
- What failures do users report that scores miss?
- What behaviors are being optimized unintentionally?
- Should any metric be retired?

Assign actions and owners. An audit without follow-through is merely an eval of the eval loop, which can continue recursively until the sun cools.

RETIRING A BENCHMARK

Retirement is healthy. Create a deprecation record:

```
suite: research_quality_v2
status: deprecated
reason:
  - 94_percent_always_pass
  - new_product_supports_multi_source_tasks
  - suspected_example_exposure
replacement: research_quality_v3
```

```
score_comparability: none  
historical_use: regression_only_for_12_months  
decision_owner: research_product  
date: 2026-08-24
```

Preserve historical results with labels. Do not splice new-suite scores into old charts. If a few cases remain valuable, migrate them with new IDs or explicit lineage.

Benchmark retirement can feel like losing progress. Retiring one proves the measurement program still has a pulse.

BEFORE YOU MOVE ON

Try to defeat your eval:

1. Produce a bad output that passes.
2. Produce a good output that fails.
3. Find one stale criterion.
4. Find one missing production category.
5. Compare public and fresh holdout performance.
6. Check judge false passes by segment.
7. Identify one offline metric never validated against a user outcome.
8. Nominate one saturated case for retirement.

The eval should emerge a little bruised and much more useful.

PART IV

Build

Owning the Loop

“Everyone owns quality” is a fine value and a terrible ticket assignee.

Evals decay when nobody owns the standard.

The harness may belong to a platform team. The rubric may need a domain expert. The task may originate in support. The grader may be written by engineering. A product leader may make the release decision. Legal or safety may define a hard boundary.

This distribution is normal. Ambiguity about it is optional.

THE SEVEN OWNERS

Eval platform

Owns shared infrastructure:

- task and result storage;
- isolated harnesses;
- model and prompt version capture;
- grader execution;
- trace viewer;
- CI and deployment integration;
- access control and audit logs;
- cost and latency observability;
- re-grading and reproducibility;
- platform reliability.

The platform team should not decide what counts as a clinically safe response or a fair refund. It makes those standards executable and observable.

Anthropic’s guidance describes a similar division: dedicated eval teams can own infrastructure while domain and product teams contribute tasks and interpret results [ANTH-01].

Product or domain owner

Owns the meaning of good for the workflow:

- prioritized capabilities and risks;
- case coverage;
- rubric intent;
- product tradeoffs;
- release thresholds;
- connection to user outcomes;
- retirement of stale behavior.

This person cannot outsource the product definition to a benchmark or a model judge.

Domain experts

Own standards that require specialized judgment:

- gold labels;
- case-specific criteria;
- adjudication;
- dangerous ambiguity;
- judge calibration;
- high-consequence audit;
- changes in domain practice.

Experts may be clinicians, lawyers, educators, researchers, support-policy specialists, security engineers, or experienced operators. “Human fallback” is too vague a job description. Give named experts defined questions.

Engineering team

Owns product behavior and strong checks:

- task fixtures;
- executable graders and invariants;
- tool contracts;
- trace instrumentation;
- fixes and regression cases;
- runtime prevention;
- harness compatibility;

- on-call response for relevant failures.

The team that changes the system should see the cases that define its success.

Statistics or data science

Owns measurement design where scale or consequence justifies specialization:

- representative sampling;
- paired comparisons;
- cluster and repeated-trial analysis;
- confidence intervals;
- power and experiment design;
- delayed-outcome modeling;
- drift detection;
- proxy validation.

Small teams may not have this role. They still need someone accountable for the questions, with external review for consequential decisions.

Safety, risk, legal, or compliance

Owns hard constraints relevant to its mandate:

- prohibited behavior;
- approval requirements;
- held-out adversarial audits;
- data handling;
- regulatory evidence;
- escalation and incident obligations;
- residual-risk review.

This function should define enforceable criteria with the product team, not arrive at the end carrying a ceremonial red pen.

Decision owner

Owns release or hold.

The decision owner reviews the evidence, applies or explicitly overrides the release contract, records residual risk, and assigns follow-up. This

may be the product owner for routine changes and a more senior accountable leader for high-risk autonomy.

Name one person. Committees advise. A decision needs a chair.

CENTRAL PLATFORM, LOCAL TRUTH

The most scalable organization centralizes mechanics and distributes meaning.

LinkedIn has described horizontal teams responsible for shared evaluation, testing, and prompt foundations alongside vertical teams building product agents [ORG-01]. Its search work also describes product-manager adjudication and large-scale judging infrastructure [ORG-02]. These layers of ownership work together.

A central team can provide:

- task schema;
- harness SDK;
- grader registry;
- trace viewer;
- CI templates;
- statistical reporting;
- access and governance;
- training and office hours.

Domain teams provide:

- real cases;
- product policies;
- rubric criteria;
- subject-matter labels;
- release decisions;
- production interpretation.

Avoid two failure shapes.

Central oracle: the eval team owns every rubric and becomes a queue. Domain meaning is lost in translation.

Local islands: every product invents schemas, judges, dashboards, and definitions. Results cannot be reproduced or compared, and infrastructure work repeats.

The interface is a service contract: the platform guarantees mechanics; the domain guarantees standards and ownership.

Agent skills make this split concrete. The capability author should own the questions, expected behaviors, fixtures, and domain-specific grader intent alongside the skill. The platform should stage isolated environments, run matched baselines across supported harnesses, normalize traces, and retain reports. ACES describes this as developer-guided evaluation with bring-your-own-task and bring-your-own-grader extension points [ACES-01]. The central system supplies a protocol; it does not confiscate the product contract.

Composition boundaries create another review job. A skill may work alone but route poorly when twenty plausible neighbors are visible. The catalog or platform owner must provide group-workspace tests and realistic decoys, while the skill author reviews whether failures reflect description, content, prerequisites, or interaction. Ownership follows the seam where the failure can be prevented.

A PRACTICAL RACI

Customize this table. Fill it with names or it is decorative.

Activity	Accountable	Responsible and consulted
Select capabilities	Product owner	Domain expert, engineering, data, risk
Create task fixtures	Engineering	Product, domain expert, eval platform
Define expert rubric	Product owner	Domain expert, data, risk
Implement deterministic grader	Engineering	Domain expert, platform, risk
Calibrate model judge	Product owner	Domain expert, data, platform, risk
Operate harness	Eval platform	Engineering, data, risk
Review production traces	Product owner	Engineering, domain expert, data, risk
Set release contract	Decision owner	Product, engineering, domain, data, risk
Approve exception	Decision owner	Product, engineering, domain, data, risk
Retire suite	Product owner	Engineering, domain, platform, data, risk

The accountable person owns the decision or outcome. The responsible and consulted group performs the work or supplies evidence. A separate detailed RACI can include informed stakeholders, but the print map stays focused on who must act.

If one cell contains “AI Council” seven times, add people.

THE OPERATING CADENCE

Evals need clocks.

Every change

- Run deterministic smoke and regression cases.
- Capture versions and paired differences.
- Block must-not-fail regressions.
- Attach artifacts to the change.

Owner: engineering, supported by the platform.

Nightly

- Run the full capability suite.
- Repeat stochastic tasks.
- Report cost, latency, and grader health.
- flag new regressions and unstable cases.

Owner: platform for execution; product engineering for response.

Weekly trace review

Sixty minutes, fixed sample, cross-functional participants.

Agenda:

1. Five minutes: volume, release, and incident context.
2. Thirty minutes: read ten to twenty sampled traces.
3. Ten minutes: review new failure categories and disagreements.
4. Ten minutes: choose cases, fixes, or grader changes.
5. Five minutes: assign names and dates.

Outputs:

- annotated traces;
- candidate eval cases;
- taxonomy updates;
- policy questions;
- runtime-prevention ideas;
- owners.

Owner: product or domain owner. Engineers and experts attend. The meeting should look at data, not slides about data.

Monthly eval health review

- Judge false-pass and false-fail audit.
- Coverage by capability, risk, and segment.

- Saturated, stale, duplicate, or leaking cases.
- Harness and grader reliability.
- Production-to-eval learning lead time.
- Proxy ledger and experiment results.
- Open release exceptions.
- Rights, privacy, and retention review.

Owner: product and eval platform jointly, with domain and risk participation.

Quarterly or major-change review

- Held-out and adversarial evaluation.
- Dataset refresh and contamination risk.
- Power and sample design.
- Policy and model dependency updates.
- Suite retirement.
- Autonomy and permission review.
- Offline-to-online validity.

Owner: decision owner for the product area.

Incident cadence

An incident creates immediate work outside the calendar:

- preserve evidence;
- create a regression case;
- repair prevention and detection;
- verify under related tasks;
- review whether release criteria change.

The regression case belongs in the incident completion criteria.

REVIEW WORK IS PRODUCT WORK

Trace annotation is often treated as leftover labor. The organization then wonders why criteria are vague and graders drift.

High-quality review requires:

- domain context;
- protected time;

- a good interface;
- clear escalation;
- feedback showing how labels changed the product;
- quality checks and calibration;
- reasonable queue sizes;
- recognition in role expectations.

LinkedIn reports using linguists at substantial daily review volume [ORG-01]. HealthBench's construction involved hundreds of physicians [HEALTH-01]. Those contributors built the measurement system.

Avoid paying annotators for speed alone. Throughput targets can reward shallow review. Track agreement, adjudication quality, evidence use, and fatigue. Rotate high-stakes queues. Sample reviewer work for coaching, not surveillance theater.

INCENTIVES CAN OPEN THE LOOP

If teams are rewarded for launch date and benchmark score but not production outcomes, they will optimize launch date and score. Morality has very little to do with it. The employee scorecard is working exactly as designed.

Balance incentives:

- feature delivery;
- hard-gate quality;
- learning lead time;
- incident recurrence;
- production outcome;
- cost per accepted result;
- stale-case closure;
- reviewer burden.

Do not reward the number of eval cases. A thousand duplicates are not better coverage. Reward important failure classes made observable and prevented.

DECISION RECORDS PRESERVE MEMORY

For every major release, keep:

- decision and owner;
- alternatives;
- eval versions;
- primary results and uncertainty;
- paired disagreements reviewed;
- hard gates;
- segment, cost, and latency results;
- known grader and coverage limitations;
- exception or residual risk;
- rollout and rollback plan;
- follow-up date;
- production validation result.

Six months later, a model will change and someone will ask why a threshold exists. The record prevents threshold folklore.

MATURITY LEVELS

Level 0: demo

Success is shown through selected examples. No durable tasks or owners.

Level 1: cases

Twenty to fifty real cases, manual review, early taxonomy. Useful for discovery.

Level 2: repeatable suite

Versioned tasks, graders, harness, baseline, regression checks.

Level 3: release system

CI gates, held-out audits, release contracts, cost and segment reporting.

Level 4: production loop

Recurring production sampling, trace review, delayed outcomes, incident-to-case process.

Level 5: adaptive measurement

Judge calibration, proxy validation, drift, benchmark retirement, clear central/local ownership.

Vendor count says nothing about maturity. A spreadsheet with a weekly owner may be Level 1. A beautiful platform nobody uses may be Level Decorative.

BEFORE YOU MOVE ON

Put actual names beside:

- platform owner;
- product-standard owner;
- domain adjudicator;
- engineering owner;
- measurement owner;
- risk owner;
- release decision owner.

Schedule the weekly trace review and monthly health review. Define their required outputs. Then calculate your current learning lead time from a recent production failure to its permanent regression case.

Anything without an owner lives in the wish pile.

The First Thirty Days

Start with twenty cases and one decision. You can purchase a platform later, after it has something useful to platform.

The first month should produce a working eval loop for one important workflow.

Not the whole company. Not every model. Not a unified theory of helpfulness. One workflow, chosen because it matters, repeats, and has outcomes the team can inspect.

At the end of thirty days, you should be able to:

- replay real tasks;
- grade important outcomes;
- compare a change with a baseline;
- inspect disagreements and traces;
- apply a release rule;
- sample production;
- assign recurring ownership.

BEFORE DAY ONE: CHOOSE THE SLICE

A good pilot workflow has:

- meaningful user value;
- repeated tasks;
- accessible traces;
- observable outcomes;
- enough failures to learn from;
- bounded action scope;
- an engaged domain owner;

- a change the team expects to make.

Avoid the easiest toy workflow. It may demonstrate the tooling without testing the method. Avoid the most consequential autonomous workflow if you have no evaluation practice. Choose a vertical slice where the team can learn safely.

Examples:

- support article answer with citations;
- refund eligibility recommendation without autonomous payment;
- coding agent for one repository;
- research report section with source checks;
- appointment-intake classification with human action;
- document extraction into a validated schema.

Write the decision:

In thirty days, we will decide whether version B should replace version A for this workflow under a limited production rollout.

Name the decision owner.

WEEK 1: DISCOVER

Day 1: instrument and sample

Confirm traces include:

- inputs;
- messages;
- tool calls and results;
- final state or artifact;
- model, prompt, tool, and policy versions;
- cost, latency, and errors;
- user feedback where allowed.

Create a mixed sample of twenty traces: ordinary, complaints, high-risk, new segments, and apparent successes. Record selection strata.

Deliverable: trace-sample-v1 with privacy review.

Day 2: shared review

Product, engineering, and a domain expert review the first five traces together. Agree on what counts as evidence. Use open notes.

Deliverable: five annotated traces and an evidence vocabulary.

Day 3: independent review

Review the remaining fifteen in pairs or independently. Mark user goal, outcome, first departure, evidence, severity, and open notes.

Deliverable: twenty annotated traces.

Day 4: taxonomy

Cluster observations. Separate user outcome, evidence, and intervention hypothesis. Identify disagreement and missing policy.

Deliverable: failure taxonomy v1 with examples and unresolved questions.

Day 5: choose tasks

Select:

- five common failures;
- five important successes;
- five high-consequence or edge cases;
- five confusing or disagreement cases.

Twenty to fifty cases are enough to start learning [ANTH-01].

Deliverable: task backlog with owner and purpose.

Week 1 checkpoint

Do not continue if the team cannot access outcome evidence or no domain owner will resolve ambiguity. Fixing instrumentation and ownership is the work, not a delay before the work.

WEEK 2: ENCODE

Day 6-7: write task contracts

For each case, define setup, input, limits, expected evidence, metadata, and review date. Redact or synthesize sensitive details.

Have someone who did not author the case attempt to find:

- a bad outcome that passes;
- a good outcome that fails.

Deliverable: task-set-v1.

Day 8: inventory truth

For every criterion, identify the strongest evidence:

- environment;
- executable test;
- structured evidence;
- deterministic rule;
- model judgment;
- expert judgment.

Mark hard gates and quality dimensions.

Deliverable: grader map.

Day 9: implement the bottom rungs

Build state checks, schema validators, invariants, and rules. Capture check-level evidence. Distinguish task failure from grader error.

Deliverable: deterministic grader suite.

Day 10: draft judgment criteria

Write narrow criteria for what remains. Have experts label a calibration sample. Record disagreements. Do not begin with an overall one-to-ten judge.

Deliverable: rubric v1 and initial gold labels.

Week 2 checkpoint

Run the harness twice on the same fixed artifacts. Deterministic graders should reproduce results. Any difference needs a reason. Confirm task, environment, and grader versions are stored.

WEEK 3: MEASURE

Day 11-12: baseline

Run current production version A over the suite. Repeat stochastic tasks at least three times if budget allows. Store task-level outcomes, traces, cost, and latency.

Deliverable: baseline run with validity report.

Day 13: calibrate judges

Run model judges on expert-labeled cases. Create confusion matrices and category breakdowns. Inspect false passes first. Narrow scope or route uncertain categories to people.

Deliverable: judge calibration report.

Day 14: implement version B

Make the product change suggested by the failure analysis. This might be a prompt change, retrieval fix, tool redesign, policy route, deterministic guard, or model swap.

Resist changing five layers at once. Change one layer so you can tell what taught you something.

Deliverable: versioned candidate B.

Day 15: paired comparison

Run A and B on the same tasks and budgets. Inspect new-only and old-only passes. Calculate uncertainty appropriate to the sample. Report hard gates, segments, cost, latency, and repeated reliability.

Deliverable: comparison report.

Week 3 checkpoint

Write the release contract before the final held-out run. If the team already saw every case while tuning B, set aside fresh cases from production or delay the claim. “Held out emotionally” does not count.

WEEK 4: OPERATE

Day 16-17: integrate the funnel

Put fast deterministic cases in local or pull-request workflows. Put broader, stochastic, and model-judged cases in nightly runs. Define invalid-run statuses and artifact retention.

Deliverable: local/PR/nightly pipeline.

Day 18: held-out review

Run fresh cases. Review must-not-fail outcomes and judge false-pass risk. Apply the release contract. The decision owner chooses release, hold, limited rollout, or more evidence.

Deliverable: signed release decision.

Day 19: production plan

Define:

- representative and risk-enriched samples;
- runtime guards;
- automated asynchronous scoring;
- human review volume;
- delayed outcomes;
- canary percentage;
- rollback rules;
- privacy and retention.

Deliverable: production-evidence plan.

Day 20: ownership and calendar

Name the seven owners. Schedule weekly trace review and monthly eval health review. Add incident-to-regression-case criteria. Record suite review dates.

Deliverable: RACI and operating calendar.

Days 21-30: limited rollout and learning

Roll out under the plan. Review early traces. Compare offline predictions with production evidence. Fix instrumentation gaps. Add representative failures. Do not spend the remaining days polishing a dashboard while traces remain unread.

Deliverables:

- production trace sample;
- new regression cases;
- first proxy-ledger entry;
- 30-day retrospective;
- next-quarter backlog.

THE THIRTY-DAY ARTIFACT SET

At completion, the repository or eval system should contain:

```
evals/  
  README.md  
  decisions/  
    objective.md  
    release-v1.md  
  tasks/  
    manifest.yaml  
    cases/  
  graders/  
    specs/  
      deterministic/  
      judge-prompts/  
  calibration/  
    gold-labels.jsonl  
    judge-report.md  
  runs/  
    baseline/  
    candidate/  
    heldout/  
  taxonomy/  
    failures-v1.md  
  production/  
    sampling-plan.md  
    review-log.md  
  ownership.md  
  CHANGELOG.md
```

The exact layout can change. Keep the artifacts.

THREE PILOT SIZES

Small team

- 20 cases;
- one engineer and one product/domain owner;
- manual trace review;
- deterministic graders plus one narrow judge;

- spreadsheet or JSONL results;
- weekly cadence.

Growing product

- 50–200 cases;
- platform support;
- CI and nightly runs;
- expert calibration;
- segment reporting;
- production sampling;
- release contracts.

Consequential domain

- risk-tiered tasks;
- independent domain labels;
- held-out adversarial audit;
- stronger runtime prevention;
- formal approval and evidence retention;
- privacy, legal, and safety review;
- controlled autonomy;
- explicit incident obligations.

Let consequence and repetition set the rigor. Fashion has enough responsibilities already.

COMMON STALLS

“We need the perfect taxonomy first”

No. Version one needs to help select tasks. It will change.

“We need hundreds of labels”

You need enough to learn and calibrate the first decision. Start with twenty to fifty tasks and expand where uncertainty matters.

“We should choose a platform”

Choose the task and evidence model first. Tools become easier to evaluate after you know the work.

"The judge agreement is only 80 percent"

Inspect by category and error cost. It may be excellent for some criteria and unusable for others.

"Production data is too sensitive"

Then build a governed redaction and synthetic-fixture path. Do not pretend public benchmarks represent your users.

"The new version's average is better"

Inspect paired regressions, hard gates, segments, cost, and reliability.

"Nobody can attend weekly review"

Then the organization has decided that learning from product behavior is lower priority than every competing meeting. Make that decision visible.

THE NEXT NINETY DAYS

After the pilot:

Month 2

- expand task coverage from production;
- improve the trace viewer;
- automate more bottom-rung graders;
- stabilize CI and nightly runs;
- validate the first offline proxy online;
- add judge drift audit.

Month 3

- onboard a second workflow using shared schemas;
- formalize central/local ownership;
- add held-out and adversarial cases;
- measure learning lead time;
- review cost per accepted outcome;
- retire weak or duplicate cases.

Quarter end

- audit coverage and rights;
- review incidents and regression memory;
- compare proxy and user outcomes;
- update autonomy and runtime controls;
- publish an internal eval health report;

- choose the next capability frontier.

BEFORE YOU MOVE ON

Put a date beside Day 1. Choose the workflow and decision owner. Book the first twenty-trace review.

The loop starts when the team agrees to look at reality on a schedule, usually one calendar invite before any evaluator runs.

The Loop Stays Open

The first eval suite will be wrong.

It will miss an important category. One task will be ambiguous. A judge will like long answers too much. The production sample will expose a policy nobody knew was still active. Someone will discover that the “final state” field is a cached summary produced before the final tool call.

Good. Start anyway.

The eval loop exists so the system can correct its idea of truth.

Read the failure, write the task, and find where evidence lives. Use the strongest available grader, then try to embarrass it. Compare the change honestly. Release under constraints. Watch what users experience. Give every artifact an owner and a date to be questioned again.

The four cases in this book show why the loop must stay open.

SWE-bench gives tests both the hero and villain roles: indispensable, precise, and sometimes wrong. τ -bench lets the database interrupt the agent’s victory speech. HealthBench puts expertise inside the definition of quality. DeepResearch Bench makes citations show their work.

No universal metric is hiding among them. They share one durable habit: move the grader toward the consequence.

Once a team works this way, its meetings change. The strongest opinion in the room loses its automatic crown. So do the newest model, the cleanest demo, and the metric with the most decimal places. The team can ask, “What evidence would change our mind?” and then build a system that collects it.

The inner loop gets safer because it can verify, retry, stop, and escalate against real signals. The outer loop gets faster because important runs leave evidence, incidents become cases, and releases can be compared with what came before.

Failure does not disappear. It becomes harder to repeat unnoticed, which is the more useful superpower.

Not bad for a Tuesday afternoon.



Appendixes

Appendix A:

Exercises

These exercises turn the book into a working eval program. They can be completed by one product team, used as workshop modules, or assigned across product, engineering, domain, data, and risk roles.

Use real product data only under appropriate privacy and access controls. If those rules keep the real data off-limits, create representative synthetic traces that preserve the failure relationship without preserving personal information.

EXERCISE 1: TWENTY TRACES, NO TAXONOMY

Purpose: Discover what your system actually gets wrong before choosing metrics.

Time: 90 minutes for the group session, plus sample preparation.

Participants: Product owner, engineer, domain expert, optional data or risk partner.

Prepare: Twenty traces: eight randomly sampled, four user complaints, four consequential actions, two new-segment cases, and two apparent successes. Record the selection stratum. Show task, messages, tool events, final state, cost, latency, and available user outcome.

Steps:

1. Review the first five together.
2. For each, write the user goal, final outcome, strongest evidence, first departure, severity, and an open note.
3. Review ten independently or in pairs.
4. Compare notes. Do not force agreement.
5. Review the final five while testing emerging labels.
6. Identify three cases: one common, one consequential, one ambiguous.

Deliverable: Annotated trace sheet plus raw open notes.

Debrief: Which apparent success failed after state or evidence inspection? Which disagreement revealed a missing policy? Which system component did reviewers blame without enough evidence?

Extension: Repeat with a representative sample and compare discovery categories with estimated prevalence. Explain why the two samples should not be combined without weights.

EXERCISE 2: BUILD A FAILURE TAXONOMY

Purpose: Convert trace observations into categories useful for engineering and product decisions.

Time: 60 minutes.

Input: Notes from Exercise 1.

Steps:

1. Put each first-departure note on a separate card.
2. Cluster cards by observed user outcome.
3. For every cluster, write the evidence that proves the failure.
4. Separately list intervention hypotheses: prompt, retrieval, tool, policy, model, controller, grader, or organization.
5. Name categories at an actionable middle level.
6. Attach concrete examples and counterexamples.
7. Mark high disagreement and must-not-fail categories.

Deliverable: Taxonomy v1 with three fields per category:

```
id: false_completion_claim
definition: response states an external action completed when
final evidence does not confirm it
evidence: final environment state or authoritative operation
status
examples: [trace-104, trace-219]
counterexample: response says action is pending after timeout
severity: high
intervention_hypotheses: [tool_semantics, retry_controller,
response_grader]
```

Debrief: Which categories describe outcomes and which accidentally describe suspected causes? Which category would two qualified reviewers apply differently? What policy question must be resolved before measurement?

Extension: Give the taxonomy to a reviewer who missed Exercise 1. Measure where they use “other” or disagree. Revise definitions rather than merely coaching the reviewer.

EXERCISE 3: TURN FIVE FAILURES INTO TASK CONTRACTS

Purpose: Preserve the diagnostic property of production failures in replayable, privacy-safe cases.

Time: 90 minutes.

Input: Five traces representing different categories.

Steps:

1. State the capability or risk each trace represents.
2. Identify the property that made the original case hard.
3. Remove names, identifiers, and incidental details.
4. Preserve the relevant policy conflict, state, ambiguity, or tool behavior.
5. Write setup, input, limits, expected evidence, owner, source, and review date.
6. Label the case capability, regression, adversarial, or calibration.
7. Have another participant try to solve the case using only the contract.

Deliverable: Five versioned task files.

Quality test: A competent person should have enough information to succeed, multiple valid solutions should remain possible where appropriate, and the expected evidence should follow from the stated contract.

Debrief: Did simplification remove the reason the case was difficult? Did the expected answer prescribe wording rather than properties? Which environment dependency needs to be frozen?

Extension: Use the SWE-bench task-fairness questions [SWE-06]. Ask one reviewer to argue that the grader is too narrow and another that it is too broad.

EXERCISE 4: BUILD THE GRADER WITHOUT AN LLM

Purpose: Move every criterion as low as possible on the grader ladder.

Time: 60 minutes.

Input: Three task contracts.

Steps:

- 1. List every claim, state transition, policy constraint, and quality criterion.
- 2. For each, ask whether environment state can answer it.
- 3. If not, try executable test, structured evidence, or deterministic rule.
- 4. Mark the judgment that remains.
- 5. Identify hard gates, quality dimensions, diagnostics, cost, and latency.
- 6. Sketch check-level output, including grader-error status.

Deliverable: Grader map.

Criterion	Evidence	Grader	Hard gate?	Known gap
Refund not issued	Payment state	State asser- tion	Yes	Delayed set- tlement
Explanation accurate	State + re- sponse	Narrow judge	Yes	Implied claims
Tone re- spectful	Response	Judge	No	Cultural variation

Debrief: Which criterion did the group initially give to a model even though code or state could answer it? Which rule is literal but being asked to infer meaning?

Extension: Redesign product output to emit one structured artifact that makes a previously expensive criterion cheap to grade.

EXERCISE 5: HACK THE GRADER

Purpose: Find false passes and false fails before optimization finds them for you.

Time: 75 minutes.

Roles: Grader author, hacker, product/domain judge.

Steps:

1. The author explains the task contract and grader outputs, not implementation secrets.
2. The hacker creates three bad outcomes intended to pass:
 - literal keyword compliance;
 - right final value through unsafe process;
 - benchmark-specific or reference-answer mimicry.
3. The hacker creates two good alternatives intended to fail.
4. Run or manually apply the grader.
5. The domain judge decides the true labels from evidence.
6. Revise task or grader. Add every successful attack as a grader regression case.

Deliverable: Adversarial grader report.

```
attack: false_refund_claim_with_pending_language
expected: fail
grader_v2: pass
cause: rule checks word "pending" anywhere in response
repair: bind claim span to authoritative state
regression_case: grader_attack_012
```

Debrief: Did the repair overfit the exact attack? What class of bad outcomes does it now catch? What independent evidence would make gaming harder?

Extension: Swap graders between teams. Outside hackers are less attached to what the author meant.

EXERCISE 6: CALIBRATE A MODEL JUDGE

Purpose: Define the scope in which an automated judge can support a decision.

Time: Two to four hours depending on labeling.

Prepare: Fifty to one hundred cases across clear passes, clear fails, ambiguity, segments, and adversarial near misses.

Steps:

1. Write one narrow criterion with pass, fail, and insufficient-evidence labels.
2. Two qualified reviewers label independently.
3. Adjudicate disagreement and record reasons.
4. Freeze development and audit splits.
5. Run the judge with supplied evidence.
6. Build a confusion matrix.
7. Calculate false-pass and false-fail rates.
8. Break results down by category, language, length, and difficulty.
9. Test position, verbosity, style, and rubric-keyword bias.
10. Define which cases route to a person.

Deliverable: Judge calibration report with a clear allowed scope.

Decision prompt: If this judge gates a consequential action, which error is more costly? Does the observed false-pass rate support that use? If it only prioritizes a human queue, how does the tradeoff change?

Extension: Change the judge model or prompt. Re-run the frozen audit split. Explain why performance on the development set is not the release estimate.

EXERCISE 7: PASS@K AND PASS^K

Purpose: Choose a reliability metric matching real use.

Time: 45 minutes.

Scenario: A research agent succeeds on 75 percent of individual tasks. The product can generate three candidates and select one for a single

report. Another workflow uses the agent once per day for eight days with no expert selection.

Steps:

1. Calculate $\text{pass}@3$ under an independence assumption.
2. Calculate pass^8 .
3. List reasons actual attempts may be correlated.
4. Add selector error: suppose the selector chooses a successful candidate 90 percent of the time when one exists.
5. Add cost: each generation costs \$0.40 and selection costs \$0.10.
6. Define product metrics for both workflows.

Deliverable: One-page reliability decision.

Debrief: Which headline would a demo prefer? Which metric would a returning user prefer? Can retries address transient tool failure but not repeated policy misunderstanding?

Extension: Use task-level trial data to classify always-pass, unstable, and always-fail cases. Decide whether to invest in retries, routing, or capability improvement.

EXERCISE 8: PAIRED RELEASE COMPARISON

Purpose: Compare two systems on the same tasks without losing case-level meaning.

Time: 60 minutes.

Scenario: On 100 tasks, both pass 62, new only passes 18, old only passes 8, and both fail 12.

Steps:

1. Calculate old and new pass rates.
2. Calculate the observed paired difference.
3. Identify the discordant case count.
4. Apply or look up the exact two-sided McNemar/binomial result; compare with the worked example in Chapter 9.
5. Inspect hypothetical categories for the 18 wins and 8 regressions.
6. Add a must-not-fail regression among the eight.

7. Make a release, hold, limited-rollout, or more-data decision.

Deliverable: Release decision with effect, uncertainty, hard gates, and follow-up.

Debrief: Why is “ p greater than .05” not the same as “no effect”? Why can one hard-gate failure outweigh ten routine wins? What minimum effect should have been written before the run?

Extension: Cluster the tasks into ten customer scenarios. Explain why task-level independence may understate uncertainty and how cluster bootstrap resampling would work.

EXERCISE 9: DESIGN PRODUCTION SAMPLING

Purpose: Connect offline evaluation to real behavior without confusing discovery samples and estimates.

Time: 75 minutes.

Steps:

1. Define the production unit: task, conversation, user, or account-period.
2. Choose a representative random sample and rate.
3. Define risk-enriched strata.
4. Define a post-release change-focused sample.
5. Identify delayed outcome events and windows.
6. List automated asynchronous graders.
7. Specify human review volume and adjudication.
8. Define privacy fields, retention, access, and redaction.
9. Describe how selected traces become tasks.

Deliverable: Production sampling plan with inclusion probabilities or clear nonrepresentative labels.

Debrief: Which sampled queue can estimate prevalence? Which is for discovery? What failures are invisible to user ratings? What data cannot leave the approved environment?

Extension: Design a ten-percent canary with rollback triggers and a randomized experiment with primary and guardrail outcomes.

EXERCISE 10: ASSIGN OWNERSHIP AND CADENCE

Purpose: Keep the eval alive after launch week.

Time: 60 minutes.

Steps:

1. Name the platform, product, domain, engineering, measurement, risk, and decision owners.
2. Complete the RACI for tasks, rubrics, graders, runs, production review, release, and retirement.
3. Schedule weekly trace and monthly health reviews.
4. Define required attendees and outputs.
5. Add incident-to-regression-case completion criteria.
6. Measure learning lead time for the last incident.
7. Identify the handoff most dependent on memory.

Deliverable: Signed ownership page and calendar invitations.

Debrief: Where did several groups believe another group was accountable? Which review has no decision to make? Which domain expert lacks protected time?

Extension: Estimate reviewer capacity. If production sampling creates 200 cases per week and experts can review 40, design automation and prioritization without silently dropping high-risk work.

EXERCISE 11: RUN THE FULL HACKER-FIXER LOOP

Purpose: Practice adversarial evolution of tasks, graders, and system design.

Time: Half day.

Teams: Hacker, fixer, domain adjudicator, release owner.

Rounds:

1. Fixer receives ten tasks and builds a system or proposed responses.
2. Hacker studies grader behavior and submits attacks.
3. Domain adjudicator labels intent from evidence.
4. Grader author repairs false passes and false fails.

5. Fixer updates the product.
6. Release owner evaluates on a hidden round.

Rules: The grader author cannot win by blocking exact attack strings. The hacker receives points for general failure classes. The fixer receives points for moving prevention below the prompt layer. The release owner reports cost and reliability alongside pass rate.

Deliverable: A changelog showing how each attack altered task, grader, runtime guard, tool interface, or product behavior.

Debrief: Did the teams converge on brittle rules or stronger evidence? Did the product learn to refuse everything? Which attacks suggest a new capability suite rather than another regression case?

EXERCISE 12: RETIRE AN EVAL SUITE

Purpose: Treat benchmark maintenance and retirement as normal life-cycle work.

Time: 75 minutes.

Input: A real or hypothetical mature suite.

Steps:

1. Calculate always-pass, unstable, and always-fail case shares.
2. Identify duplicates, stale policy dependencies, suspected leakage, and weak graders.
3. Compare current production failures with suite coverage.
4. Separate cases worth retaining as regressions.
5. Design replacement capability cases.
6. Write score-comparability rules.
7. Create a deprecation and communication plan.

Deliverable: Benchmark deprecation record and migration map.

Debrief: Is the suite saturated because the capability is solved or because graders are weak? Can historical results remain visible without implying comparability? Which protected cases should never be published?

Extension: Use the SWE-bench Verified lifecycle as a comparison [SWE-02] [SWE-03]. Identify curation, audit, contamination, and retirement practices your internal suite lacks.

Appendix B:

Templates

Copy these templates into the system where your team already works. Markdown, YAML, JSONL, a database, and an eval platform can all support the method. The nonnegotiable properties are versioning, ownership, evidence, and review.

TEMPLATE 1: EVALUATION OBJECTIVE AND DECISION RECORD

```
# Evaluation Objective: [workflow / release]

## Decision
We will decide whether to [release / choose / route / automate / retire] _____.

Decision owner: [name]
Decision date: [date]

## Population and unit
Target population: _____
Unit of evaluation: [response / conversation / task / user-period] _____
Important segments: _____

## Product outcome
The user or business outcome we actually care about is _____.
The offline eval is expected to predict it because _____.

## Primary evidence
Primary metric: _____
Minimum worthwhile effect or threshold: _____
Uncertainty method: _____

## Hard gates
- [must-not-fail criterion]
- [must-not-regress criterion]

## Tradeoffs
Cost limit: _____
Latency limit: _____
Escalation or refusal limit: _____

## Known limitations
- _____
```

Follow-up

Production validation: _____

Review date: _____

TEMPLATE 2: TASK CASE

```
id: workflow_capability_001
version: 1
status: active
purpose:
  capability: ""
  failure_category: ""
  decision: ""
source:
  type: production_redacted | synthetic | public_benchmark |
incident
  reference: ""
  rights_class: internal_restricted | original | licensed |
anti_leak
setup:
  environment_version: ""
  policy_version: ""
  initial_state: {}
  tools: []
  permissions: []
input:
  messages: []
limits:
  max_turns: 0
  max_tool_calls: 0
  max_wall_seconds: 0
  max_cost_usd: 0
  retry_policy: ""
expected:
  hard_gates: []
  quality_criteria: []
  evidence_required: []
metadata:
  severity: low | medium | high | must_not_fail
  suite: capability | regression | adversarial | calibration
  tags: []
  owner: ""
  created: YYYY-MM-DD
  last_reviewed: YYYY-MM-DD
  review_by: YYYY-MM-DD
  dependencies: []
```

TEMPLATE 3: TRACE ANNOTATION SHEET

```
trace_id: ""
reviewer: ""
review_date: YYYY-MM-DD
```

```

selection:
  stratum: random | complaint | high_risk | new_segment |
  apparent_success | other
  inclusion_probability: null
task:
  user_goal: ""
  contract_summary: ""
outcome:
  final_result: success | partial | failure | unknown
  world_state: ""
  strongest_evidence: ""
analysis:
  first_departure_step: ""
  observation: ""
  recovery_behavior: ""
  severity: low | medium | high | must_not_fail
  user_recoverable: true
taxonomy:
  user_outcome_category: ""
  evidence_category: ""
  intervention_hypotheses: []
grader:
  false_pass: false
  false_fail: false
  missing_criterion: ""
next:
  candidate_eval_case: true
  runtime_prevention: ""
  owner: ""
open_note: ""

```

TEMPLATE 4: FAILURE TAXONOMY REGISTER

```

# Failure Taxonomy v[version]

Owner: [name]
Effective date: [date]
Supersedes: [version]

## [category_id]: [name]

Definition: [observable user or system outcome]

In scope:
- [example]

Out of scope:
- [counterexample]

Strongest evidence:
- [state / event / source / expert criterion]

Severity guidance:
- Low: _____

```

- Medium: _____
- High: _____
- Must-not-fail: _____

Known intervention hypotheses:

- [prompt / retrieval / tool / controller / policy / grader / organization]

Example traces: [IDs]

Related eval cases: [IDs]

Open disagreements: _____

Review trigger: _____

TEMPLATE 5: GRADER SPECIFICATION

```
id: grader_name
version: 1
owner: ""
criterion: ""
decision_supported: ""
grader_type: environment | executable | structured | rule |
model_judge | expert
inputs:
  required: []
  optional: []
output:
  schema: {}
  labels: []
  grader_error_status: "invalid"
interpretation:
  hard_gate: false
  pass_rule: ""
  threshold: null
evidence:
  pointer_required: true
  independent_from_agent: ""
calibration:
  dataset: ""
  last_run: YYYY-MM-DD
  false_pass_rate: null
  false_fail_rate: null
known_failures:
  - ""
adversarial_cases:
  - ""
operations:
  expected_cost_usd: 0
  expected_latency_ms: 0
  timeout_status: invalid
  sandbox: ""
review_trigger:
  - grader_model_change
  - criterion_change
```

- production_drift
- scheduled_audit

TEMPLATE 6: MODEL-JUDGE PROMPT

```
SYSTEM
You grade exactly one criterion. Use only the TASK CONTRACT,
EVIDENCE, and
OUTPUT. Do not infer missing facts from outside knowledge. If
the evidence is
insufficient, return insufficient_evidence. Return valid JSON
only.

CRITERION
[One binary or categorical requirement, including important
exceptions.]

LABELS
pass: [definition]
fail: [definition]
insufficient_evidence: [definition]

TASK CONTRACT
{{task_contract}}

EVIDENCE
{{evidence}}

OUTPUT TO GRADE
{{output}}

RETURN
{
  "label": "pass | fail | insufficient_evidence",
  "output_span": "exact relevant span or empty",
  "evidence_span": "exact relevant evidence or empty",
  "reason": "one sentence tied to the criterion"
}
```

Keep the actual prompt, examples, model version, inference settings, and parser version in the grader registry. Do not put protected benchmark examples in a generally distributed prompt library.

TEMPLATE 7: JUDGE CALIBRATION REPORT

```
# Judge Calibration: [grader ID and version]

Criterion: _____
Decision supported: _____
Expert owner: _____
Judge model/prompt: _____
```

Date: _____

Dataset

Cases: _____

Source strata: _____

Development / calibration / audit split: _____

Languages and segments: _____

Adjudication process: _____

Confusion matrix

		Expert pass		Expert fail	
	---	----		----	
	Judge pass				
	Judge fail				

Agreement: _____

Pass precision: _____

Pass recall: _____

False-pass rate: _____

False-fail rate: _____

Insufficient-evidence rate: _____

Breakdown

	Category / segment		n		false pass		false fail		decision	
	---	----		----		----		----		----

Bias and attack tests

- Position swap: _____
- Length control: _____
- Style control: _____
- Rubric keyword attack: _____
- Long-context evidence position: _____

Allowed use

The judge may _____.

The judge may not _____.

Cases routed to people: _____.

Follow-up

Next audit: _____

Owner: _____

TEMPLATE 8: EVAL RUN COMPARISON

Eval Comparison: [A] vs [B]

Decision: _____

Run date: _____

Valid run: yes / no

Invalid tasks or grader errors: _____

Versions

```

| Artifact | A | B |
|---|---|---|
| Code / prompt | | |
| Model / settings | | |
| Tools / policy | | |
| Task set | | |
| Graders | | |
| Harness | | |

```

Primary paired results

```

| | Count |
|---|---|
| Both pass | |
| A only | |
| B only | |
| Both fail | |

```

Observed paired difference: _____
 Confidence interval / analysis: _____
 Minimum worthwhile effect: _____

Hard gates

- _____

Reliability

Trials per task: _____
 pass@k: _____
 pass^k: _____
 Unstable tasks: _____

Segments

[table]

Cost and latency

Cost per attempt: _____
 Cost per accepted task: _____
 Median / p95 latency: _____

Discordant-case review

New wins: _____
 Regressions: _____
 New failure classes: _____

Limitations

- _____

TEMPLATE 9: RELEASE DECISION

Release Decision: [release]

Decision owner: _____
 Date: _____

Decision: release / hold / limited rollout / gather more evidence

Pre-specified contract

Primary threshold: _____

Hard gates: _____

Segment rule: _____

Cost rule: _____

Latency rule: _____

Evidence

Run comparison: [link]

Held-out audit: [link]

Judge calibration: [link]

Reviewed trace IDs: _____

Result against contract

[table of each rule and pass/fail/invalid]

Known limitations and residual risk

- _____

Exception

Contract overridden? yes / no

Reason and authority: _____

Compensating controls: _____

Expiry / follow-up: _____

Rollout

Initial percentage: _____

Guardrails: _____

Rollback trigger: _____

Rollback owner: _____

Production review date: _____

TEMPLATE 10: PRODUCTION SAMPLING PLAN

```
workflow: ""
owner: ""
effective_date: YYYY-MM-DD
unit: task | conversation | user | account_period
representative_sample:
  method: ""
  rate: 0
  weights_recorded: true
risk_samples:
  - name: consequential_action
    trigger: ""
    review_rate: 1.0
  - name: grader_disagreement
    trigger: ""
    review_rate: 1.0
change_focused_sample:
  release: ""
```

```

    affected_segments: []
    automated_graders: []
    human_review:
      cases_per_week: 0
      reviewers: []
      adjudicator: ""
    delayed_outcomes:
      - event: ""
        window_days: 0
    privacy:
      retained_fields: []
      redacted_fields: []
      retention_days: 0
      access_role: ""
      external_judge_allowed: false
    feedback_loop:
      case_creation_owner: ""
      weekly_review: ""

```

TEMPLATE 11: EVAL SUITE HEALTH REVIEW

Eval Health Review: [suite] [month/quarter]

Owners present: _____

Coverage

- Capabilities missing: _____
- High-risk categories missing: _____
- Thin segments: _____
- Production failures without cases: _____

Grader health

- False-pass / false-fail audit: _____
- High-disagreement criteria: _____
- Successful grader attacks: _____
- Judge drift: _____

Dataset health

- Always-pass cases: _____
- Unstable cases: _____
- Stale dependencies: _____
- Duplicates: _____
- Leakage / contamination risk: _____
- Rights or retention issues: _____

Operations

- Harness invalid rate: _____
- Grader error rate: _____
- Median incident-to-case lead time: _____
- Open release exceptions: _____

Product validity

- Offline metrics validated online: _____
- Proxies that failed: _____

- New user failure categories: _____

Actions

Action	Owner	Due	Completion evidence
----	----	----	----

TEMPLATE 12: OWNERSHIP PAGE

Eval Ownership: [workflow]

Responsibility	Named owner	Backup	Cadence / trigger
----	----	----	----
Eval platform			
Product standard			
Domain adjudication			
Engineering behavior			
Sampling/statistics			
Safety/risk/privacy			
Release decision			

Weekly trace review: [day/time, owner, required output]

Monthly health review: [day/time, owner, required output]

Quarterly held-out review: [owner]

Incident-to-regression criterion: _____

Escalation path: _____

TEMPLATE 13: DATASET PROVENANCE AND RIGHTS CARD

```
artifact_id: ""
artifact_type: task | trace | annotation | screenshot | dataset
| diagram_adaptation
source:
  title: ""
  url: ""
  author_or_org: ""
  version_or_commit: ""
  accessed: YYYY-MM-DD
rights:
  license: ""
  third_party_content: true
  permission_required: unknown
  permission_record: ""
restrictions:
  anti_contamination_request: false
  contains_gold_answer: false
  contains_personal_data: false
  publication_allowed: false
allowed_use:
  internal_eval: true
  print_description: true
  verbatim_reproduction: false
```

```

    screenshot: false
    notes: ""
    owner: ""
    review_by: YYYY-MM-DD

```

TEMPLATE 14: BENCHMARK VERSION AND DEPRECATION LOG

```

suite: ""
version: ""
status: active | maintenance | deprecated | retired
effective_date: YYYY-MM-DD
reason: []
indicators:
  always_pass_rate: null
  production_coverage_gap: ""
  leakage_risk: ""
  grader_validity: ""
replacement: ""
case_migration:
  regression_only: []
  revised_ids: []
score_comparability:
  compatible_with_previous: false
  explanation: ""
historical_reporting_label: ""
decision_owner: ""
communication: ""

```

TEMPLATE 15: PROXY LEDGER

```

# Offline-to-Online Proxy Ledger

| Offline metric | Intended product outcome | Mechanism | Last
validation | Result | Decision | Owner | Next review |
|---|---|---|---|---|---|---|
| | | | | keep / revise / retire | | |

```

For each entry, attach the experiment or observational analysis and record important segment differences. A proxy with no validation date is a hypothesis, not an outcome metric.

Appendix C: Case Study Field Guide

These four cases come with public artifacts worth opening: datasets, task schemas, annotation guidance, grader implementations, harness code, and benchmark-maintenance decisions.

This appendix explains what to study and how to adapt the methods without copying protected evaluation items into a product or publication.

SWE-BENCH: REAL REPOSITORIES AND EXECUTABLE CONTRACTS

What the project gives you

SWE-bench begins with real GitHub issues and corresponding changes from twelve Python repositories. The original benchmark contains 2,294 tasks [SWE-01]. Its public repository and evaluation harness show how to provision repository environments, apply predicted patches, execute task-specific tests, and aggregate results [SWE-04] [SWE-05].

SWE-bench Verified adds a particularly valuable editorial artifact: human annotation instructions used to assess whether a task is clear, solvable, and fairly tested [SWE-06]. The Verified curation screened 1,699 candidates with 93 Python developers and three independent reviews per candidate, retaining 500 [SWE-02].

The benchmark's later history supplies a second set of artifacts: an audit of often-failed tasks, a contamination analysis, and a decision to stop using the suite as a primary reported measure [SWE-03]. Few case studies show creation, curation, scaling, auditing, and retirement this clearly.

What to inspect

1. **Task representation.** Identify how repository, base commit, problem statement, tests, and expected patch lineage are stored.
2. **Environment isolation.** Follow the path from task to container image, repository checkout, patch application, and test execution.

3. **Test selection.** Examine how fail-to-pass and pass-to-pass behavior is represented. Ask what broader regression coverage remains outside a task.
4. **Run artifact.** Find where prediction, logs, exit status, and per-task result are preserved.
5. **Annotation rubric.** Compare task clarity and test fairness questions with your own task review.
6. **Versioning.** Identify which result claims depend on dataset release and harness configuration.

Internal adaptation

For an internal coding-agent eval, create a task package:

```
id: billing_issue_042
repository: billing-service
base_commit: 6f12a9b
problem_statement: "Refund events may be applied twice after
worker restart."
environment_image: billing-eval:2026-08-24
visible_checks:
  - unit/refunds_idempotency_test.py
trusted_checks:
  - integration/restart_replay_test.py
invariants:
  - one_ledger_entry_per_idempotency_key
limits:
  wall_minutes: 30
  network: disabled
grader_version: billing_patch_v3
```

Then conduct two independent reviews:

- A maintainer asks whether the problem statement contains the required context.
- A tester attempts an alternative valid repair and asks whether trusted checks accept behavior rather than a preferred implementation.

Preserve ordinary regression tests in addition to task-specific tests. A patch that fixes the issue while breaking neighboring behavior should not pass the product eval.

What not to conclude

A benchmark score does not equal general software-engineering ability. Task languages, repository selection, environment, issue style, available tools, time budget, and contamination all matter. A test pass does not establish maintainability, security, or alignment with unstated intent.

The targeted 138-task audit in [SWE-03] should not be quoted as the defect rate of all SWE-bench Verified tasks. It selected tasks frequently failed across o3 runs. Its value is diagnostic: even a heavily curated executable benchmark needs continuing audit.

Publication and reuse

The repository is MIT licensed, but repository tasks can embed material originating in upstream projects. Record the exact license and commit for anything reproduced. Public gold patches or hidden tests should not be copied into prompts, training data, or general-audience examples. Newly written schematic patches and tests are safer for teaching.

T-BENCH AND T³-BENCH: GRADING STATE, POLICY, AND INTERACTION

What the project gives you

The original τ -bench paper defines tool-agent-user tasks for realistic customer-service domains and introduces pass^k as a repeated-reliability measure [TAU-01]. The maintained repository, currently named tau2-bench and branded τ^3 -bench, expands domains, simulation modes, task fixes, evaluator components, and trajectory workflows [TAU-02].

Open the evaluator implementation. It makes composite grading concrete by combining checks over environment state, actions, communicated information, and natural-language assertions [TAU-03]. The CLI documentation shows how to run, inspect, and re-grade trajectories [TAU-04].

What to inspect

1. **Domain policy.** Observe how policy is made available to the agent and represented in tasks.
2. **Database or environment.** Identify initial and expected states and how a run is reset.

3. **Tools.** Inspect schemas, error behavior, and the relationship between tool call and state.
4. **User simulation.** Determine how user goals and responses create interaction variability.
5. **Evaluator composition.** Trace how each grader produces evidence and how the task result is aggregated.
6. **Trajectory review.** Inspect the artifacts required to explain a fail.
7. **Repeated trials.** Compare individual task success with pass^k .

Internal adaptation

Build one tool-agent task around an uncertain side effect:

```
id: order_cancel_timeout_009
initial_state:
  order_status: paid
  refund_status: none
tool_behavior:
  first_cancel_call:
    side_effect: order_cancelled_refund_pending
    response: timeout
user_goal: cancel_order
policy:
  confirmation_required: true
expected:
  order_status: cancelled
  refund_status: pending
  cancel_event_count: 1
communication:
  must_not_claim: refund_completed
```

Grade four layers:

- state matches the intended outcome;
- one cancellation event exists;
- confirmation occurred before commitment;
- communication distinguishes pending from completed.

Run at least three trials. A system that occasionally handles the timeout correctly but often duplicates the action has a reliability failure even if its best trace is excellent.

Version caution

The maintained benchmark has corrected many tasks. Do not mix original paper scores, current task files, and a new evaluator under one label.

Identify release, split, commit, model, attempts, and simulator configuration.

Publication and reuse

Confirm repository and generated-trajectory rights before reproducing a full trace. For teaching, original traces like the timeout example above preserve the method without creating leakage or confusing historical versions.

HEALTHBENCH: EXPERT-DEFINED, CASE-SPECIFIC QUALITY

What the project gives you

HealthBench contains 5,000 health conversations and 48,562 case-specific criteria created with 262 physicians across 60 countries [HEALTH-01]. It includes multilingual, multi-turn, synthetic, and human-adversarial material. The evaluation approach scores criteria individually with assigned values rather than relying on one generic impression.

The meta-evaluation code compares automated grading with physician opinion, making the judge itself an object of study [HEALTH-04]. Consensus and hard variants help distinguish broad agreement from difficult cases.

What to inspect without copying examples

The dataset card documents schema, fields, license, and an explicit request not to reproduce evaluation examples in plain text or images [HEALTH-03]. Honor the request. Study:

1. how criteria attach to cases;
2. how points and pass conditions are represented;
3. how automated grading is compared with experts;
4. how categories and agreement are reported;
5. how variants represent consensus and difficulty.

Do not paste a benchmark conversation into internal documents, slides, screenshots, this book, or a model prompt merely because the file is accessible.

Internal adaptation

Use a nonmedical or appropriately expert-reviewed original case. For example, a security assistant receives a log indicating a likely credential leak. Case-specific criteria might require it to:

- identify that an exposed production credential is urgent;
- advise revocation rather than only rotation in a future deploy;
- preserve relevant audit evidence;
- avoid asking the user to paste the secret;
- distinguish containment from complete incident resolution;
- escalate to the incident process.

Two security experts label fifty outputs independently. Disagreements reveal ambiguous incident policy. After adjudication, a narrow model judge grades each criterion. Report false passes by criterion; a judge suitable for “mentions revocation” may be unsuitable for “does not destroy forensic evidence.”

This adaptation preserves the method: case-specific domain criteria, expert ownership, automated scale, and judge meta-evaluation.

What not to conclude

High judge agreement does not establish clinical safety in a different product. A benchmark may cover many scenarios without representing a local population, language mix, workflow, regulatory context, or action boundary. Use domain-specific evaluation and human accountability appropriate to the product.

Publication and reuse

Legal license and responsible publication are separate decisions. The dataset is MIT licensed, but the anti-contamination request is clear. Describe structure and results, link to official sources, and create original analogous material.

DEEPRESEARCH BENCH: REPORT QUALITY AND EVIDENCE QUALITY

What the project gives you

DeepResearch Bench provides 100 expert-written tasks across 22 fields, 400 reports from four systems, and 150 expert RACE annotations. Its

authors report balancing domains using analysis of 96,147 user queries and involving more than 70 master’s-level or domain-expert annotators, with three annotators per task in the consistency study [DRB-01].

The public dataset includes tasks, reports, and annotations under Apache 2.0 [DRB-02]. The repository contains the evaluation implementation [DRB-03].

The benchmark separates:

- **RACE:** comprehensiveness, depth, instruction following, readability;
- **FACT:** citation accuracy and effective citation count.

What to inspect

1. **Task distribution.** How are domains and user-query patterns represented?
2. **Report artifact.** What structure, citation format, and metadata are available?
3. **RACE criteria.** How are report-level dimensions operationalized?
4. **FACT pipeline.** How are citations resolved and support evaluated?
5. **Human annotations.** What evidence accompanies labels and where do annotators disagree?
6. **Judge implementation.** Which prompts, models, and aggregation choices affect the score?

Internal adaptation

Build a ten-task research set from real, permissioned work. Each task should specify audience, scope, source constraints, recency, and output requirements. Grade a structured artifact:

```
{
  "report": "...",
  "claims": [
    {
      "claim": "...",
      "importance": "major",
      "citations": [
        {"url": "...", "source_id": "...", "passage": "..."}
      ]
    }
  ]
}
```

Use separate checks:

- URL resolves;
- source meets date and type constraints;
- cited passage appears in the source;
- passage supports the claim;
- major factual claims have effective citations;
- report covers required subquestions;
- synthesis distinguishes evidence from inference;
- readability and instruction following meet the audience contract.

Give each check an evidence pointer. The claim–passage judge should see only the local claim, passage, and necessary context. An expert editor reviews sampled false passes and source-quality judgments.

Third-party rights

Apache 2.0 on a dataset does not erase copyright in papers, news articles, or other sources cited inside reports. Before reproducing report passages, citations, screenshots, or annotations, review the individual material. For book examples, original claims and short invented source passages are safer.

COMPARING THE FOUR CASES

Design question	SWE-bench	τ -bench / τ^3	Health-Bench	DeepResearch Bench
Primary artifact	Code patch	Tool interaction	Health response	Research report
Strong truth	Tests/repo state	Environment state	Expert criteria	Claim/source relationship
Key grader risk	Wrong or narrow tests	Right state via wrong process	Judge-expert disagreement	Citation presence mistaken for support
Reliability concern	Repeat across tasks/runs	pass^k	Category and expert agreement	Judge and report variability
Maintenance lesson	Audit and retirement	Task/version fixes	Protect cases from leakage	Separate dimensions and rights

Do not crown one case as the universal model. Borrow their strongest habits:

- executable checks where possible;
- final-state and action evidence for tools;
- case-specific expert criteria for judgment;
- local claim–evidence checks for research;
- repeated-trial reliability;
- versioned tasks and graders;
- explicit anti-contamination and rights practices;
- benchmark audit and retirement.

That combination is the practical eval loop.

Appendix D:

Statistical Recipes

This appendix is a desk reference, not a substitute for statistical review in high-consequence work. Every recipe begins with a defined unit, sample, and decision. Run calculations in reviewed code and preserve the inputs with the eval result.

RECIPE 1: BINOMIAL PASS RATE WITH WILSON INTERVAL

Use when: Each independent unit has a binary pass/fail result and the sample design supports treating units as independent.

Inputs: Passes x , total valid units n , confidence level.

Estimate:

$$\hat{p} = x / n$$

For a 95 percent Wilson interval with $z = 1.96$ [STAT-02]:

```
denominator = 1 + z2/n
center = ( $\hat{p} + z^2/(2n)$ ) / denominator
half_width = z/denominator × sqrt( $\hat{p}(1-\hat{p})/n + z^2/(4n^2)$ )
interval = center ± half_width
```

Worked example: $x = 43$, $n = 50$, $\hat{p} = .86$. Wilson interval is approximately .738 to .930.

Report: “43/50 tasks passed (86%; Wilson 95% CI 73.8–93.0%).”

Do not use without adjustment when: Tasks are clustered, trials repeat the same task, selection is failure-enriched, or labels have important uncertainty.

RECIPE 2: PAIRED BINARY COMPARISON

Use when: Old and new systems run on the same tasks under comparable conditions.

Build:

	New pass	New fail
Old pass	both pass	old only
Old fail	new only	both fail

The observed difference is:

$$(new_only - old_only) / total_tasks$$

For a simple exact McNemar/binomial test, condition on discordant pairs $d = new_only + old_only$ and test whether new-only wins follow $Binomial(d, .5)$ [STAT-03].

Worked example: New only 18, old only 8, total 100. Difference is +10 points. Exact two-sided p-value is approximately 0.0755.

Report: Effect size, uncertainty or test, all four cells, and categories among discordant cases. Do not report only the p-value.

Decision note: A must-not-fail regression can block release regardless of aggregate improvement.

RECIPE 3: CLUSTER BOOTSTRAP FOR A PAIRED DIFFERENCE

Use when: Tasks share a customer, document, conversation, repository, scenario, or other source of correlated failure.

Procedure:

1. Choose the cluster level before analysis.
2. Keep old/new results paired within each task.
3. Sample clusters with replacement until the bootstrap sample has the original number of clusters.
4. Include all tasks from each sampled cluster, or apply the planned within-cluster resampling design.
5. Calculate paired pass-rate difference.
6. Repeat many times, such as 5,000 or 10,000.
7. Use appropriate quantiles for an interval and inspect the bootstrap distribution.

Report: Number of clusters and tasks, resampling procedure, repetitions, observed effect, and interval.

Caution: Five clusters remain weak evidence even if they contain thousands of questions. More rows do not manufacture more independent worlds.

Anthropic's guidance provides further examples and cautions for clustered eval data [STAT-01].

RECIPE 4: REPEATED STOCHASTIC TASKS

Use when: Model sampling, user simulation, tools, or control paths vary.

For task i , run r trials and estimate:

$$\hat{p}_i = \text{passes}_i / r$$

Report across tasks:

- mean and median $\hat{p}_i$;
- always-pass share;
- unstable share (both pass and fail observed);
- always-fail share;
- cost and latency distribution;
- user-relevant sequence reliability.

Do not pool all trials and present them as independent tasks. Preserve the task hierarchy.

If comparing variants, use matched seeds or simulated-user configurations where that meaningfully reduces noise, but do not claim deterministic comparability when external tools vary.

RECIPE 5: PASS@K

Use when: The product generates k candidates and needs at least one success.

Under a simplified independent per-attempt probability p :

$$\text{pass}@k = 1 - (1 - p)^k$$

For code-generation benchmarks that sample n candidates and observe c correct, use the appropriate finite-sample estimator described by Chen et al. rather than substituting the simplistic formula [STAT-04].

Also measure:

- selector success given a correct candidate exists;
- total generation and grader cost;
- latency under parallel or sequential generation;
- correlation among candidates;
- fraction of tasks where every candidate fails the same way.

Overall product success is not $\text{pass}@k$ if the selector cannot identify the good candidate.

RECIPE 6: PASS^k

Use when: The user needs k consecutive tasks or sessions to succeed.

Under a simplified independent probability p :

$$\text{pass}^k = p^k$$

At $p = .75$:

- $\text{pass}^3 = 42.19\%$
- $\text{pass}^8 = 10.01\%$

τ -bench foregrounds this repeated-reliability view [TAU-01].

Caution: Dependence matters. Stable task-specific weaknesses can make sequence success lower or differently distributed. Estimate user- or scenario-level reliability from grouped data when available.

RECIPE 7: JUDGE CONFUSION MATRIX

Use when: A model judge is compared with accepted expert labels.

Define “pass” as the positive label:

- TP: judge pass, expert pass;
- FP: judge pass, expert fail—the false passes;
- FN: judge fail, expert pass;
- TN: judge fail, expert fail.

Calculate:

```
pass_precision = TP / (TP + FP)
pass_recall = TP / (TP + FN)
false-pass_rate = FP / (FP + TN)
false-fail_rate = FN / (FN + TP)
```

Track insufficient-evidence and grader-error outcomes separately rather than forcing them into pass or fail.

Break the matrix down by criterion, severity, language, length, and ambiguity. Choose thresholds and routing based on error cost.

RECIPE 8: STRATIFIED PRODUCTION ESTIMATE

Use when: Production is sampled at different rates by segment or risk stratum.

For mutually exclusive strata h , estimate each stratum rate $\hat{p}_h$ and weight by its share W_h in the target production population:

$$\hat{p} = \sum W_h \hat{p}_h$$

Record selection probabilities and current production stratum sizes. Calculate uncertainty using a method matching the sampling design.

Do not treat a queue that contains every complaint and one in a thousand ordinary runs as a simple random sample. Its raw failure proportion is a review workload statistic, not a production rate.

RECIPE 9: COST PER ACCEPTED OUTCOME

Use when: Variants differ in model cost, retries, tool use, grading, or human review.

```
total_cost =
    generation_cost
    + tool_cost
    + retry_cost
    + automated_grader_cost
    + human_review_cost
    + failed_run_overhead

cost_per_accepted_outcome = total_cost / accepted_outcomes
```

State how human time is valued and which infrastructure costs are included. Report cost per attempt and acceptance rate too; the decomposition explains movement.

Pair with median and p95 latency. A cheap task that takes five minutes may have unusual product economics.

RECIPE 10: SIMULATION-BASED POWER

Use when: The release rule combines pairing, clusters, repeated trials, hard gates, or several thresholds.

Procedure:

1. Fit or specify a plausible data-generating process from pilot results.
2. Include task difficulty, cluster variation, trial noise, grader error, and missing runs as relevant.
3. Simulate results under candidate true effects.
4. Apply the exact planned release rule.
5. Repeat many times.
6. Estimate how often the rule releases under each effect and how often hard-gate violations occur.
7. Vary assumptions.

Deliverable: A table of true effect, sample design, release probability, expected cost, and key assumptions.

Simulation does not remove assumptions. It makes them executable and discussable.

REPORTING CHECKLIST

Every quantitative eval report should state:

- decision and pre-specified rule;
- unit of evaluation;
- target population;
- sampling and enrichment;
- tasks, clusters, and trials;
- invalid or missing results;
- model, prompt, tool, grader, and harness versions;

- numerator, denominator, estimate, and uncertainty;
- paired disagreements for comparisons;
- hard-gate outcomes;
- segment results;
- cost and latency;
- grader validity;
- limitations not captured by the interval;
- production follow-up.

If the report cannot state these yet, label the result exploratory. Exploratory is a respectable word. It has prevented many charts from being promoted beyond their abilities.

Glossary

Terms are defined as used in this book.

Accepted outcome. A task result that passes required gates and is accepted for product use. The denominator in cost per accepted outcome. A generated response is not accepted merely because generation completed.

Action event. A structured record of an attempted or completed tool action, including arguments, authorization, timestamps, result, error, idempotency key, and relevant postcondition.

Adjudication. Resolution of reviewer disagreement by a named person or process qualified to define the product or domain standard. Good adjudication repairs ambiguous criteria instead of forcing a label and walking away.

Agent loop, or inner loop. The runtime cycle in which an agent observes state, decides, acts, verifies, and continues, stops, or escalates.

Annotation. A human-created label and supporting evidence attached to a task, output, or trace. High-quality annotation records why the label applies and where evidence lives.

Benchmark. A defined collection of tasks, harnesses, graders, and reporting procedures used to compare systems. A benchmark result is meaningful only with version, procedure, budget, and limitations.

Benchmark retirement. The documented removal or demotion of a suite because it is stale, saturated, contaminated, invalid, or no longer aligned with product behavior.

Calibration. Comparison of a grader or confidence signal against labels or outcomes accepted as the relevant standard, followed by adjustment of criteria, prompts, thresholds, or scope.

Calibration set. Labeled data used to choose judge prompts, thresholds, or operating scope. It is distinct from the held-out audit set used to estimate performance after calibration.

Capability case. A task representing an important ability the product should develop or compare. Capability cases may be deliberately challenging and need not all pass today.

Capability suite. A broad collection of cases used to measure the frontier of useful product behavior, compare architectures, and guide investment. It differs from the stable regression suite.

Case-specific criterion. A grading requirement written for the facts and risks of one task rather than a generic quality dimension. Health-Bench is a prominent published example of case-specific rubric design [HEALTH-01].

Cluster. A group of observations sharing a source of correlated error, such as one customer, conversation, document, repository, policy, or generated scenario.

Codex. This project's complete editorial registry of sources, claim boundaries, versions, use locations, and reproduction cautions. The codex is more exhaustive than the reader-facing reference list.

Confidence interval. A range produced by a statistical procedure intended to express sampling uncertainty around an estimate. It does not include every source of uncertainty, such as grader bias, dataset shift, or harness bugs.

Contamination. Exposure of a model or development process to evaluation tasks, answers, gold patches, or close equivalents, weakening the interpretation of measured performance.

Criteria drift. Change in the operational meaning of good as reviewers see outputs, resolve disagreement, or encounter new policy and user behavior. Criteria drift should be versioned and reviewed rather than denied.

Decision owner. The named person accountable for release, hold, limited rollout, or other decision supported by an eval.

Deterministic grader. A grader that returns the same result for the same inputs under a controlled environment, such as a schema check, invariant, rule, or executable test.

Discovery sample. A deliberately varied or failure-enriched sample used to learn what can go wrong. Its raw proportions do not estimate ordinary production prevalence.

Discordant pair. In a paired A/B eval, a task on which one system passes and the other fails. Discordant cases supply the direct evidence of change.

Effective citation. A citation that resolves to an acceptable source and supports the claim attached to it. Citation presence alone is not effectiveness.

Environment oracle. Authoritative state or outcome exposed by the task environment and used as grading evidence, such as a reservation record, payment event, test result, or simulator state.

Eval. A repeatable procedure for collecting evidence about system behavior to support a decision. It includes tasks, a harness, graders, and reporting—not only a metric.

Eval funnel. The progression from fast local checks to pull-request regression gates, nightly capability runs, held-out audits, and controlled production validation.

Eval loop, or outer loop. The product-improvement cycle of observing production, analyzing errors, updating tasks and graders, comparing changes, releasing, and learning from new outcomes.

Executable truth. Evidence produced by running a test, invariant, validator, or program against an artifact or state. Executable truth is conditional on the correctness and completeness of the executable contract.

Expert truth. A standard requiring qualified domain judgment, especially for ambiguous, value-laden, or consequential criteria. Expert truth may be scaled through calibrated automated graders but remains expert-owned.

False fail. A grader rejects an output that the accepted standard labels as passing. Sometimes called a false negative depending on label convention.

False pass. A grader approves an output that the accepted standard labels as failing. False passes are often the critical error when an eval gates consequential release.

First departure. The earliest evidence-backed point in a trace where the run moved away from a good path or lost important information. It is often more actionable than the final visible error.

Gold label. A label accepted as the calibration or audit standard after qualified review and, where needed, adjudication. “Gold” describes its role, not infallibility.

Grader. A function that converts run evidence into a structured result for one or more criteria.

Grader attack. A deliberately constructed good output that fails or bad output that passes, used to test measurement validity.

Grader error. Failure of the measurement mechanism—timeout, parse error, missing evidence, crash, or invalid environment—rather than failure of the system being evaluated.

Grader ladder. The progression used in this book: environment oracle, executable test, structured evidence, deterministic rule, model judge, and expert judgment. A criterion should climb only as high as necessary.

Grader stack. Several graders composed to cover hard gates, quality dimensions, diagnostics, cost, and latency while preserving distinct results.

Hard gate. A criterion whose failure blocks task success or release rather than being averaged with other dimensions.

Harness. The controlled system that sets up tasks, runs agents, enforces budgets and permissions, captures traces and final state, invokes graders, and stores reproducible results.

Held-out audit. Evaluation on cases not used to tune the system or grader, typically performed before an important release or during periodic review.

Idempotency key. A stable identifier allowing repeated requests to represent one intended side effect. It helps a system retry safely after uncertain tool responses.

Insufficient evidence. A legitimate grader result indicating the supplied artifacts cannot resolve the criterion. It should not silently default to pass or fail.

Invariant. A property that must remain true across all valid agent paths, such as no duplicate charge, no negative balance, or every side effect has authorization.

Judge drift. Change in the error behavior of a model judge caused by model, prompt, rubric, evidence, language, or production-distribution changes.

Learning lead time. Elapsed time from a meaningful production failure to an implemented and verified product change with a durable regression case or monitoring improvement.

Measurement sample. A representative or statistically designed sample used to estimate rates for a defined population.

Metamorphic test. A test defined through a transformation that should preserve or predictably change behavior when no single reference answer exists.

Minimum worthwhile effect. The smallest improvement large enough to change a product decision. It should be written before examining comparison results.

Model judge. A model prompted to classify or score another system's output under a rubric and supplied evidence. It must be calibrated for a defined scope.

Outcome chain. The hypothesized connection from an offline criterion to near-term behavior and an ultimate product or user outcome.

Paired evaluation. Comparison of variants on the same tasks, environments, and budgets so task difficulty is shared and discordant cases can be inspected.

pass@k. The probability or estimator describing at least one success among k candidates. It fits generate-and-select use cases and must include selector quality and cost.

pass k . The probability that all k uses succeed. It is a repeated-reliability metric introduced prominently in τ -bench [TAU-01].

Postcondition. Authoritative evidence describing state after an action, used to confirm whether the intended side effect occurred.

Production escape. A failure not detected by offline or runtime controls before affecting users or real state.

Proxy. A measurable signal used in place of a delayed or expensive product outcome. A proxy should have an explicit outcome hypothesis and validation history.

Proxy divorce. Loss of a useful relationship between an offline metric and the product outcome it was intended to predict.

Regression case. A durable task representing behavior that previously worked or a failure that has been repaired. Regression cases protect earned capability.

Release contract. A pre-specified decision rule covering primary effect, hard gates, segments, reliability, cost, latency, rollout, and rollback.

Representative sample. A sample selected with a known design to estimate behavior in a target population. It differs from a risk-enriched review queue.

Risk-enriched sample. A sample that deliberately oversamples consequential actions, complaints, tool errors, low-confidence runs, or new segments to improve discovery and review.

Rubric. A set of criteria and label definitions used by human or automated graders. Useful rubrics tie labels to observable evidence.

Saturation. A benchmark state where most relevant systems are near the ceiling or cases no longer differentiate useful capability.

Selector. The mechanism that chooses among several generated candidates in a pass@k workflow. Overall success depends on generation and selector error.

State truth. Evidence from authoritative environment state rather than the agent’s verbal description of an outcome.

Task contract. The full definition of an eval case: purpose, setup, input, execution limits, expected evidence, metadata, ownership, and version.

Trace. The structured evidence trail of a run, including task, messages, tool events, state, control decisions, artifact, grader results, cost, latency, and versions.

Trace viewer. An interface that places contracts, events, state, grader evidence, versions, and annotations together so teams can review behavior and create cases efficiently.

Unit of evaluation. The item counted as one observation, such as a complete task, conversation, user session, or account-period.

Version lineage. Recorded dependency among task, policy, environment, agent, grader, and harness versions, allowing a result to be reconstructed and compared honestly.

Wilson interval. A confidence interval for a binomial proportion with better small-sample behavior than the basic normal approximation [STAT-02].

References

Source IDs used throughout the book resolve here. The complete living source registry—including claim boundaries, versions, manuscript locations, and reproduction cautions—is maintained in `codex.md`.

CORE EVALUATION PRACTICE

[FOUND-01] Husain, Hamel. “AI Product Engineering.” <https://hamel.dev/notes/llm/ai-product-engineering/>.

[FOUND-02] Husain, Hamel. “A Field Guide to Rapidly Improving AI Products.” <https://hamel.dev/blog/posts/field-guide/>.

[FOUND-03] Husain, Hamel. “Your AI Product Needs Evals.” <https://hamel.dev/blog/posts/evals/>.

[JUDGE-01] Husain, Hamel. “Creating a LLM-as-a-Judge That Drives Business Results.” <https://hamel.dev/blog/posts/llm-judge/>.

[JUDGE-02] Shankar et al. “Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences.” UIST 2024. <https://arxiv.org/abs/2404.12272>.

[OPS-01] Xia et al. “Evaluation-Driven Development and Operations of LLM Agents.” 2024. <https://arxiv.org/abs/2411.13768>.

[OPS-02] Braintrust. “What is LLM evaluation?” <https://www.braintrust.dev/articles/llm-evaluation-guide>.

[ANTH-01] Anthropic. “Demystifying evals for AI agents.” <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>.

[STAT-01] Anthropic. “A statistical approach to model evaluations.” <https://www.anthropic.com/research/statistical-approach-to-model-evals>.

[OPENAI-01] OpenAI. “Evaluation best practices.” <https://developers.openai.com/api/docs/guides/evaluation-best-practices>.

[OPENAI-02] OpenAI. “Graders.” <https://developers.openai.com/api/docs/guides/graders>.

[OPENAI-03] OpenAI. “Agent evals.” <https://developers.openai.com/api/docs/guides/agent-evals>.

[ACES-01] Kevin, Christopher, et al. “Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills.” arXiv:2608.20614v1, 2026. <https://arxiv.org/abs/2608.20614>.

[ACES-02] NVIDIA. NVIDIA/SkillEvaluator. <https://github.com/NVIDIA/SkillEvaluator>.

SWE-BENCH

[SWE-01] Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. <https://arxiv.org/abs/2310.06770>.

[SWE-02] OpenAI. “Introducing SWE-bench Verified.” <https://openai.com/index/introducing-swe-bench-verified/>.

[SWE-03] OpenAI. “Why we no longer evaluate SWE-bench Verified.” <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>.

[SWE-04] SWE-bench. “Evaluation Harness.” <https://www.swebench.com/SWE-bench/api/harness/>.

[SWE-05] Princeton NLP. princeton-nlp/SWE-bench. <https://github.com/princeton-nlp/SWE-bench/blob/main/README.md?plain=1>.

[SWE-06] OpenAI. “SWE-bench Verified Annotation Instructions.” <https://cdn.openai.com/introducing-swe-bench-verified/swe-b-annotation-instructions.pdf>.

[SWE-07] Princeton NLP. SWE-agent/SWE-agent. <https://github.com/SWE-agent/SWE-agent>.

T-BENCH AND T³-BENCH

[TAU-01] Yao et al. “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.” 2024. <https://arxiv.org/abs/2406.12045>.

[TAU-02] Sierra Research. `sierra-research/tau2-bench` (current τ^3 -bench repository). <https://github.com/sierra-research/tau2-bench>.

[TAU-03] Sierra Research. Composite evaluator implementation. <https://github.com/sierra-research/tau2-bench/blob/main/src/tau2/evaluator/evaluator.py>.

[TAU-04] Sierra Research. “CLI Reference.” <https://github.com/sierra-research/tau2-bench/blob/main/docs/cli-reference.md>.

HEALTHBENCH

[HEALTH-01] “HealthBench: Evaluating Large Language Models Towards Improved Human Health.” 2025. <https://arxiv.org/abs/2505.08775>.

[HEALTH-02] OpenAI. “HealthBench.” <https://openai.com/index/healthbench/>.

[HEALTH-03] OpenAI. `openai/healthbench` dataset card. <https://huggingface.co/datasets/openai/healthbench>.

[HEALTH-04] OpenAI. `healthbench_meta_eval.py`, `openai/simple-evals`. https://github.com/openai/simple-evals/blob/main/healthbench_meta_eval.py.

DEEPRESEARCH BENCH

[DRB-01] “DeepResearch Bench: A Comprehensive Benchmark for Deep Research Agents.” 2025. <https://arxiv.org/abs/2506.11763>.

[DRB-02] `muset-ai/DeepResearch-Bench-Dataset`. <https://huggingface.co/datasets/muset-ai/DeepResearch-Bench-Dataset>.

[DRB-03] `Ayanami0730/deep_research_bench`. https://github.com/Ayanami0730/deep_research_bench.

ADDITIONAL BENCHMARKS

[PAPER-01] “PaperBench: Evaluating AI’s Ability to Replicate AI Research.” 2025. <https://arxiv.org/abs/2504.01848>.

[REBENCH-01] “RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts.” 2024. <https://arxiv.org/abs/2411.15114>.

[BROWSE-01] “BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents.” 2025. <https://arxiv.org/abs/2504.12516>.

[AGENTLENS-01] “AgentLens.” 2026. <https://arxiv.org/abs/2605.12925>. Candidate trace source; license and stable dataset access must be verified before reproduction.

STATISTICAL METHODS

[STAT-02] Wilson, Edwin B. “Probable Inference, the Law of Succession, and Statistical Inference.” *Journal of the American Statistical Association* 22(158), 1927, 209–212. <https://doi.org/10.1080/01621459.1927.10502953>.

[STAT-03] McNemar, Quinn. “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages.” *Psychometrika* 12, 1947, 153–157. <https://doi.org/10.1007/BF02295996>.

[STAT-04] Chen et al. “Evaluating Large Language Models Trained on Code.” 2021. <https://arxiv.org/abs/2107.03374>.

ORGANIZATIONAL AND PRODUCTION CASES

[ORG-01] LinkedIn Engineering. “Musings on Building a Generative AI Product.” <https://www.linkedin.com/blog/engineering/generative-ai/musings-on-building-a-generative-ai-product>.

[ORG-02] LinkedIn Engineering. “Reimagining LinkedIn’s Search Stack.” <https://www.linkedin.com/blog/engineering/search/reimagining-linkedins-search-stack>.

[ORG-03] Husain, Hamel. “Evals in Production.” <https://hamel.dev/notes/llm/ai-product-engineering/evals-production.html>.

[ORG-04] Spotify Engineering. “Better Experiments with LLM Evals: A Funnel, Not a Fork.” 2026. <https://engineering.atspotify.com/2026/5/better-experiments-with-llm-evals-a-funnel-not-a-fork>.

[ORG-05] Uber Engineering. “Introducing the Prompt Engineering Toolkit.” <https://www.uber.com/en-HK/blog/introducing-the-prompt-engineering-toolkit/>.

SOURCE AND REUSE NOTE

All numerical claims should be checked against the current linked source before final publication. Repository-based implementation claims should be pinned to a commit. HealthBench and BrowseComp examples must not be reproduced because their maintainers request protection against contamination. Public data licenses do not automatically grant rights to third-party material embedded in benchmark tasks, reports, citations, or screenshots.

About the Author

Brenn Hill is a software engineer, engineering leader, and published author based in Berlin. He graduated into the Great Financial Crisis and took the only job he could find: working at a boutique advertising agency where half the staff still cut pictures with X-Acto knives for print layouts.

Brenn has spent almost two decades building and leading engineering teams. He has worked across front-end, back-end, DevOps, data science, and data engineering while leading teams across America, Europe, and Asia.

He teaches AI-augmented development practices to engineering teams and builds open-source projects, management frameworks, and developer tools. His work focuses on the systems around AI output: verification, agent loops, human oversight, and the operating practices that turn impressive demonstrations into dependable products.

The Eval Loop is his third book.